Branch Prediction

Branch prediction is a mechanism whereby a processor uses heuristics to determine the most probable result of a branch condition. The processor places instructions that are predicted to execute into the pipeline for immediate processing after the branch. If the processor guesses incorrectly, it must remove the predicted code from the pipeline, losing all work performed on that code. This case is slightly worse than the alternative, which is to wait until the result of a branch to fill the pipeline. However, if the processor guesses correctly, performance increases, because the processor can continue to execute instructions immediately after the branch. To achieve high performance using branch prediction, the processor must contain accurate branch prediction units, which also contribute to hardware complexity.[97,98]

On-Chip Floating-Point and Vector Processing Support

Many modern processors contain on-chip execution units called coprocessors. Designers optimize coprocessors to perform specific operations that the general-purpose arithmetic and logic units (ALUs) execute slowly. These include floating point coprocessors and vector coprocessors. Vector coprocessors execute vector instructions, which operate on a set of data, applying the same instruction to each item in the set (e.g., adding one to each element in an array). Placing these coprocessors on the processor's chip decreases the communication latency between the processor and its coprocessors, dramatically increasing the speed with which these operations are performed, but also increases hardware complexity."

Additional Infrequently-Used Instructions

Perhaps the most significant divergence from the RISC philosophy is the expansion of the instruction sets in today's so-called RISC processors. These ISAs tend to include any instruction that enhances overall performance, regardless of the hardware complexity required to execute that instruction.[100] For example, the Apple PowerMac G5 processor, referred to by many as a RISC processor, contains over a hundred more instructions than its ancestor, the G3. These instructions typically manipulate large (i.e., 128-bit) integers or perform the same instruction on multiple data units, a technique called vector processing.[101]

CISC convergence to RISC

As RISC hardware has become more complex, CISC processors have adopted components of the RISC philosophy. For example, today's CISC processors often contain an optimized core subset of commonly used instructions that are decoded and executed quickly to enable performance comparable to that of RISC when complex instructions are not employed. In fact, the Intel Pentium 4 decodes all instructions into simple, fixed-size **micro-ops** before sending them to the execution unit. Often, the only reason that some complex instructions remain in a CISC ISA is to provide backward compatibility with code written for older versions of a CISC processor.[102] In this way, CISC processors incorporate the benefits of simpler instructions advocated by the RISC philosophy.

Self Review

1. Despite increased additional hardware complexity, what is the primary characteristic that distinguishes today's RISC processors from CISC processors?
2. (T/F) RISC processors are becoming more complex while CISC processors are becoming simpler.

Ans: **1)** Most of today's RISC processors continue to provide instructions of uniform length that require a single clock cycle to execute. **2)** False. RISC processors are indeed becoming more complex. Although CISC processors incorporate RISC design philosophies, their hardware complexity continues to increase.

14.8.4 Explicitly Parallel Computing (EPIC)

Techniques such as superscalar design, deep pipelines and OOO enable post-RISC processors to exploit parallelism. The hardware required to implement these features while accounting for dependencies across multiple execution units can become prohibitively expensive. In response, designers at Intel and Hewlett Packard proposed a new design philosophy called **Explicitly Parallel Instruction Computing (EPIC).** EPIC attempts to simplify processor hardware to enable a high degree of parallelism. The EPIC philosophy requires that the compiler, not hardware, determines which instructions can be performed in parallel. This technique exploits **instruction-level parallelism (ILP),** which refers to sets of machine instructions that can be executed in parallel (i.e., the instructions in the set do not rely on one another to execute).

To support ILP, EPIC employs a variation of the **very long instruction word (VLIW)** method.[103] In the VLIW method, the compiler determines which instructions should be executed in parallel. This simplifies hardware, allowing more room for execution units; the Multiflow computer, the first VLIW machine, had 28 execution, which is substantially more than modern superscalar processors.[104] However, some dependencies are known only at execution time (e.g., because of branch instructions), but VLIWs form the execution path before the program executes. This limits the level of parallelism VLIW designs can exploit.[105]

EPIC borrows from both VLIW processor designs and superscalar processor designs. EPIC processors assist the compiler by providing predictability—no out-of-order execution is used, allowing EPIC compilers to optimize the most likely execution path or paths. However, it is often difficult for a compiler to optimize even-path of execution in a program. If the path of execution is incorrectly predicted, the processor ensures program correctness (e.g., by checking data dependencies).[106]

Typical RISC and CISC processors employ branch prediction to probabilistically determine the result of a branch. Instead, EPIC processors execute all possible instructions that could follow a branch in parallel and use only the result of the correct branch once the **predicate** (i.e., the branch comparison) is resolved.[107] This technique is called **branch predication.**

Figure 14.4 illustrates the difference between program execution in an EPIC processor versus a post-RISC processor. In Fig. 14.4(a), the EPIC compiler has pro-

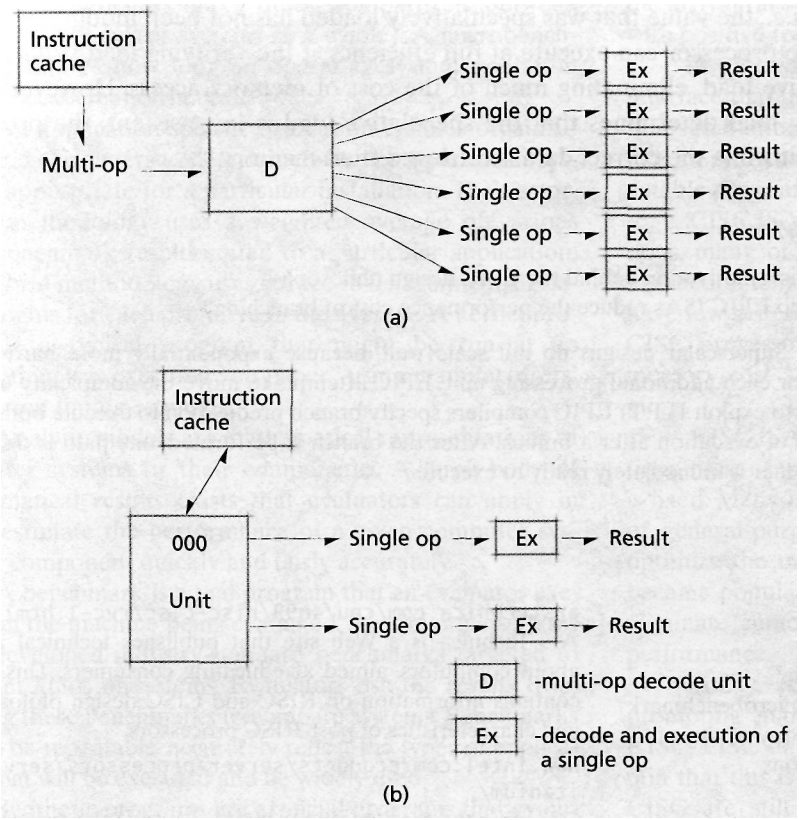

Figure 14.4 | *Instruction execution in an (a) EPIC processor and (b) post-RISC processor.*

duced a **multi-op instruction,** which is a package of a number of instructions that can be executed in parallel; the number depends on the processor. The processor decodes this multi-op into a number of single operations, then executes each simultaneously on several execution units. Because EPIC processors assist the compiler by providing predictability guarantees, they do not reorder multi-op instructions, and no OOO unit is needed. In Fig. 14.4(b), the post-RISC processor analyzes the instruction stream that it is executing and reorders instructions to find two instructions that can execute simultaneously. The processor places these two instructions into its two execution units (using a superscalar architecture) for simultaneous execution.

Aside from improving performance through parallelism, EPIC processors employ **speculative loading,** a technique that attempts to reduce memory latency. When optimizing program code, the compiler converts each load instruction to a speculative load operation and a verifying load operation. A speculative load retrieves from memory data specified by an instruction that has yet to be executed. Therefore, the processor can execute a speculative load well before the actual data

is needed. The verifying load ensures that the data from the speculative load is consistent (i.e., the value that was speculatively loaded has not been modified in memory). The processor can execute at full efficiency if the verifying load validates the speculative load, eliminating much of the cost of memory access. However, if the verifying load determines that the speculative load is inconsistent, the processor must wait while the correct data is retrieved from memory[108]

Self Review

1. What is a motivation behind the EPIC design philosophy?
2. How do EPIC ISAs reduce the performance cost of branching?

Ans: **1)** Superscalar designs do not scale well, because exponentially more hardware is needed for each additional processing unit. EPIC attempts to move the complexity into the compiler to exploit ILP. **2)** EPIC compilers specify branch predication to execute both possible paths of execution after a branch. After the branch is performed, one path is discarded and the other is immediately ready to execute.

Web Resources

`www.opersys.com/LTT/`
Provides a trace toolkit for Linux.

`www.eecs.harvard.edu/~vino/perf/hbench/`
Provides information on hbench, which is a microbenchmark suite. The site includes a link to download a copy of the suite, documentation and related technical publications.

`www.specbench.org`
SPEC is an organization that develops standard benchmarks. Its Web site describes a number of benchmarks (called SPECmarks) used to evaluate a variety of systems and publishes the results of tests with these benchmarks on real systems.

www.veritest.com/benchmarks/default.asp?visitor=
VeriTest provides computer system evaluation services. This page from its Web site provides excellent information on several commercially available benchmarks and synthetic programs.

`arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html`
Ars Technica is a Web site that publishes technical articles about computers, aimed at educating consumers. This article contains information on RISC and CISC design philosophies and characteristics of post-RISC processors.

www.intel.com/products/server/processors/server/Itanium/
Describes the Intel Itanium processor, one of the first commerdaily available EPIC processors.

`www.bapco.com`
BAPCo is an organization that develops standard benchmarks, such as the popular SYSMark for processors. Its Web site describes its benchmarks in detail.

www.linuxjournal.com/article.php?sid=2396
Describes performance monitoring in Linux.

Summary

The performance of a system depends heavily on its hardware, operating system and the interaction between the two. Cumbersome software causes poor performance, even on systems with powerful hardware, so it is important to consider a system's software performance as well as its hardware performance.

Three common purposes for performance evaluation are selection evaluation, performance projection and performance monitoring. Some common performance measures are turnaround time, response time, system reaction time, variance in response time, throughput and capacity. Some performance results may be deceptive if the evaluation does not use a representative workload or focuses on a small part of the system.

A trace is a record of system activity—typically a log of user and application requests to the operating system. A profile records the activity of a system when executing in kernel mode. These techniques are useful in evaluating systerns whose workload depends heavily on the system's execution environment.

Timings are useful for performing quick comparisons in hardware. Timings measure how many instructions a sys-

tem can execute per second. Similarly, microbenchmarks permit evaluators to make quick comparisons between operating systems (or systems as a whole). A microbenchmark measures how long an operating system operation (e.g., process creation) takes.

An application-specific evaluation enables organizations and consumers to determine whether a particular system is appropriate for a particular installation. The vector–sed methodology uses a weighted average of various microbenchmark results suited to a particular application. The hybrid methodology uses a trace to determine the relative weights for each primitive in the average. A kernel program is a typical program that might be run at an installation; it is executed "on paper" using manufacturers' instruction timing estimates.

Analytic models are mathematical representations of computer systems or their components. A large body of mathematical results exists that evaluators can apply in order estimate the performance of a given computer system or component quickly and fairly accurately.

A benchmark is a real program that an evaluator executes on the machine being evaluated. Several organizations have developed industry-standard benchmarks targeted to different kinds of systems. Evaluators use the results from running these benchmarks to compare systems. Benchmarks should be repeatable, accurately reflect the types of applications that will be executed and be widely used.

Synthetic programs are artificial programs that evaluate a specific component of an operating system; they might be constructed to match the instruction frequency distribution of a large set of programs. Synthetic programs are useful in development environments; as new features become available, synthetic programs can be used to test that these features are operational.

Simulation is a technique in which an evaluator develops a computerized model of the target system. The simulation is run to determine how a system might perform when it has been built.

Performance monitoring is the collection and analysis of system performance information for existing systems. A resource becomes a bottleneck when it hinders the progress of the system because it cannot do its job efficiently. A resource becomes saturated when it has no excess capacity to fulfill new requests. In a feedback loop, information is reported to the system about how saturated (or underutilized) a resource is. With negative feedback, the arrival rate of requests at that resource might decrease; with positive feedback, the arrival rate might increase.

A processor's instruction set architecture (ISA) is an interface that describes the processor, including its instruction set, number of registers and memory size. The ISA is like an API that low-level software uses to construct executable programs.

CISC ISAs tend to include a large number of instructions, many of which require multiple cycles to execute; instruction length is variable. Many CISC implementations have few general-purpose registers and complex hardware. CISC processors became popular because they reduced memory cost and facilitated assembly-language programming.

RISC ISAs tend to include a small number of instructions, most of which execute in one cycle; instruction length is fixed. Many RISC implementations have a large number of general-purpose registers and simple hardware. They optimize the most-common instructions. RISC processors became popular because they use pipelines efficiently and eliminate cumbersome hardware, both of which improve performance.

Today, RISC and CISC designs are converging, prompting many to call these modern processors post-RISC, FISC or second-generation RISC. Still, some maintain that this is not the case and that the terms RISC and CISC are still important. This convergence stems from increased hardware complexity and extra infrequently used instructions in RISC processors and from the fact that CISC processors often include an optimized core of RISC-like instructions.

EPIC, a newer processor-design philosophy, attempts to address the limited scalability of superscalar designs. EPIC processors place the responsibility on the compiler to determine the path of execution and use their many execution units to support a high degree of parallelism. EPIC processors also reduce the penalty for main memory accesses by performing speculative loads before execution of a load instruction; the processor performs a verifying load when the instruction is executed, ensuring data integrity. If the data that was speculatively loaded has not changed, then no action is taken as a result of the verifying load; otherwise, the processor reexecutes the instruction using data from the verifying load.

Key Terms

absolute performance measure—Measure of the efficiency with which a computer system meets its goals, described by an absolute quantity such as the amount of time in which a system executes a certain benchmark. This con-

trasts with relative performance measures such as ease of use, which only can be used to make comparisons between systems.

analytic model—Mathematical representation of a computer system or component of a computer system for the purpose of estimating its performances quickly and relatively accurately.

application vector—Vector that contains the relative demand on operating system primitives by a particular application, used in an application-specific performance evaluation.

arrival rate—Rate at which new requests are made for a resource.

benchmark—Real program that an evaluator executes on the system being evaluated to determine how efficiently the system executes that program; benchmarks are used to compare systems.

BIPS (billion instructions per second)—Unit commonly used to categorize the performance of a particular computer; a rating of one BIPS means a processor can execute one billion instructions per second.

bottleneck—Resource that hinders the progress of the system because it cannot do its job efficiently.

branch penalty—Performance loss in pipelined architectures associated with a branch instruction; this occurs when a processor cannot begin processing the instruction after the branch until the processor knows the outcome of the branch. The branch penalty can be reduced by using delayed branching, branch prediction or branch predication.

branch predication—Technique used in EPIC processors whereby a processor executes all possible instructions that could follow a branch in parallel and uses only the result of the correct branch once the predicate (i.e., the branch comparison) is resolved.

branch prediction—Technique whereby a processor uses heuristics to determine the most probable result of a branch in code; when the processor predicts correctly, performance increases, because the processor can continue to execute instructions immediately after the branch.

Business Application Performance Corporation (BAPCo)—Organization that develops standard benchmarks, such as the popular SYSMark for processors.

capacity—Measure of the maximum throughput a system can attain, assuming that whenever the system is ready to accept more jobs, another job is immediately available.

complex instruction set computing (CISC)—Processor-design philosophy, emphasizing expanded instruction sets that incorporate single instructions that perform several operations.

delayed branching—Optimization technique for pipelined processors in which a compiler places directly after a branch an instruction that must be executed whether or not the branch is taken; the processor begins executing this instruction while determining the outcome of the branch.

Dhrystone—Classic synthetic program that measures how effectively an architecture runs systems programs.

dispersion—Measure of the variance of a random variable.

distribution of response times—Set of values describing the response times for jobs in a system and the relative frequencies with which those values occur.

ease of use—Measure of the comfort and convenience associated with system use.

event-driven simulator—Simulator controlled by events that are made to occur according to probability distributions.

expected value—Sum of a series of values each multiplied by its respective probability of occurrence.

Explicitly Parallel Instruction Computing (EPIC)—Processor-design philosophy whose goals are to provide a high degree of instruction-level parallelism, reduce processor hardware complexity and improve performance.

family of computers—Series of computers that are compatible in that they can run the same programs.

fast instruction set computing (FISC)—Term describing the processor-design philosophy resulting from the convergence of RISC and CISC design philosophies. The FISC design philosophy stresses inclusion of any construct that improves performance.

feedback loop—Technique in which information about the current state of the system can influence the number of requests arriving at a resource (e.g., positive and negative feedback loops).

Hartstone—Popular synthetic benchmark used to evaluate real-time systems.

hbench microbenchmark suite—Popular microbenchmark suite, which enables evaluators to effectively analyze the relationship between operating system primitives and hardware components.

hybrid methodology—Performance evaluation technique that combines the vector-based methodology with trace data to measure performance for applications whose behavior depends strongly on user input.

Imbench microbenchmark—Microbenchmark suite that enables evaluators to measure and compare system performance on a variety of UNIX platforms.

instruction-level parallelism (ILP)—Parallelism that permits two machine instructions to be executed at once. Two instructions exhibit ILP if the execution of one does not affect the outcome of the other (i.e., the two instructions do not depend on each other).

instruction set—Set of machine instructions a processor can perform.

instruction set architecture (ISA)—Interface exposed by a processor that describes the processor, including its instruction set, number of registers and memory size.

IOStone—Popular synthetic benchmark that evaluates file systems.

kernel program—Typical program that might be run at an installation; it is executed "on paper" using manufacturers' instruction timings and used for application-specific performance evaluation.

mean—Average of a set of values.

micro-op—Simple, RISC-like instruction that is the only type of instruction processed by a Pentium processor; the Pentium's instruction decoder converts complex instructions into a series of micro-ops.

microbenchmark—Performance evaluation tool that measures the speed of a single operating system operation (e.g., process creation).

microbenchmark suite—Program that consists of a number of microbenchmarks, typically used to evaluate many important operating system operations.

MIPS (million instructions per second)—Unit commonly used to categorize the performance of a particular computer; a rating of one MIPS means a processor can execute one million instructions per second.

MobileMark—Popular benchmark for evaluating systems installed on mobile devices developed by Business Application Performance Corporation (BAPCo).

multi-op instruction—Instruction word used by an EPIC system in which the compiler packages a number of smaller instructions for the processor to execute in parallel.

negative feedback—Data informing the system that a resource is having difficulty servicing all requests and the processor should decrease the arrival rate for requests at that resource.

out-of-order execution (OOO)—Technique in which a processor analyzes a stream of instructions and dynamically reorders instructions to isolate groups of independent instructions for parallel execution.

performance monitoring—Collection and analysis of system performance information for existing systems; the information includes a system's throughput, response times, predictability, bottlenecks, etc.

performance projection—Estimate of the performance of a system that does not exist, useful for deciding whether to build that system or to modify an existing system's design.

positive feedback—Data informing the system that a resource has excess capacity, so the processor can increase the arrival rate for requests at that resource.

predicate—Logical decision made on a subject (e.g., a branch comparison).

predictability—Measure of the variance of an entity, such as response time. Predictability is particularly important for interactive systems, where users expect predictable (and short) response times.

production program—Program that is run regularly at an installation.

profile—Record of kernel activity taken during a real session, which indicates the operating system functions that are used most often and should therefore be optimized.

random variable—Variable that can assume a certain range of values, where each value has an associated probability.

reduced instruction set computing (RISC)—Processor-design philosophy that emphasizes small, simple instructions sets and optimization of the most-frequently used instructions.

response time—In an interactive system, the time from when a user presses an *Enter* key or clicks a mouse until the system delivers a final response.

saturation—Condition of a resource that has no excess capacity to fulfill new requests.

script-driven simulator—Simulator controlled by data carefully designed to reflect the anticipated environment of the simulated system; evaluators derive this data from empirical observations.

selection evaluation—Analysis regarding whether obtaining a computer system or application from a particular vendor is appropriate.

service rate—Rate at which requests are completed by a resource.

simulation—Performance evaluation technique in which an evaluator develops a computerized model of a system being evaluated. The model is then run to reflect the behavior of the system being evaluated.

SPECmark—Standard benchmark for testing systems; SPECmarks are published by the Standard Performance Evaluation Corporation (SPEC).

speculative loading—Technique whereby a processor retrieves from memory data specified by an instruction that has yet to be executed; when the instruction is executed, the processor performs a verifying load to ensure the data's consistency.

stability—Condition of a system that functions without error or significant performance degradation.

Standard Application (SAP) benchmarks—Popular benchmarks used to evaluate a system's scalability.

Standard Performance Evaluation Corporation (SPEC)—Organization that develops standard, relevant benchmarks (called SPECmarks), which are used to evaluate a variety of systems; SPEC publishes the results of tests with these benchmarks on real systems.

STREAM—Popular synthetic benchmark which tests the memory subsystem.

superscalar architecture—Technique in which a processor contains multiple execution units so that it can execute more than one instruction in parallel per clock cycle.

synthetic benchmark—Another name for a synthetic program.

synthetic program—Artificial program used to evaluate a specific component of a system or constructed to mirror the characteristics of a large set of programs.

SYSmark benchmark—Popular benchmark for desktop systems developed by Business Application Performance Corporation (BAPCo).

system reaction time—Time from when a job is submitted to a system until the first time slice of service is given to that job.

system tuning—Process of making fine adjustments to a system based on performance monitoring to optimize the system's execution for a specific operating environment.

system vector—Vector containing the results of microbenchmarks for a number of operating system primitives for a specific system, used in an application-specific evaluation.

throughput—Work-per-unit-time performance measurement.

timing—Raw measure of an isolated hardware performance metric, such as a BIPS rating, used for quick comparisons between systems.

TIPS (trillion instructions per second)—Unit used to categorize the performance of a particular computer; a rating of one TIPS means a processor can execute one trillion instructions per second.

trace (performance evaluation)—Record of real system activity, which is executed on systems to test how the system handles a sample workload.

Transaction Processing Performance Council (TPC) benchmarks—Popular benchmarks which target database systems.

turnaround time—Time from when a job is submitted until the system completes executing it.

utilization—Fraction of time that a resource is in use.

validate a model—To demonstrate that a computer model is an accurate representation of the real system the model is simulating.

variance in response times—Measure of how much individual response times deviate from the mean response time.

vector-based methodology—Method of calculating an application-specific evaluation of a system based on the weighted average of the microbenchmark results for the target system's primitives; the weights are determined by the the target application's relative demand for each primitive.

very long instruction word (VLIW)—Technique in which a compiler chooses which instructions a processor should execute in parallel and packages them into a single (very long) instruction word; the compiler guarantees that there are no dependencies between instructions that the processor executes at the same time.

WinBench 99—Popular synthetic program used extensively today in testing a system's graphics, disk and video subsystems in a Microsoft Windows environment.

WebMark—Popular benchmark for Internet performance developed by Business Application Performance Corporation (BAPCo).

Whetstone—Classic synthetic program which measures how well systems handle floating point calculations, and has thus been helpful in evaluating scientific programs.

workload—Measure of the amount of work that has been submitted to a system; evaluators determine typical workloads for a system and evaluate the system using these workloads.

Exercises

14.1 Explain why it is important to monitor and evaluate the performance of a system's software as well as its hardware.

14.2 When a user logged in, some early timesharing systems printed the total number of logged-in users.

a. Why was this information useful to the user?

b. In what circumstances might this not have been a useful indication of load?

c. What factors tended to make this a highly reliable indication of system load on a timesharing system that supported many users?

14.3 Briefly discuss each of the following purposes for performance evaluation.

a. selection evaluation

b. performance projection

c. performance monitoring

14.4 What is system tuning? Why is it important?

14.5 Distinguish between user-oriented and system-oriented performance measures.

14.6 What is system reaction time?

Is it more critical to processor-bound or I/O-bound jobs? Explain your answer.

14.7 In discussing random variables, why can mean values sometimes be deceiving?

What other performance measure is useful in describing how closely the values of a random variable cluster about its mean?

14.8 Why is predictability such an important attribute of computer systems?

In what types of systems is predictability especially critical?

14.9 Some commonly used performance measures follow.

i. turnaround time

ii. throughput

iii. response time

iv. workload

v. system reaction time

vi. capacity

vii. variance of response times

viii. utilization

For each of the following, indicate which performance measure(s) is (are) described.

a. the predictability of a system

b. the current demands on a system

c. a system's maximum capabilities

d. the percentage of a resource in use

e. the work processed per unit time

f. turnaround time in interactive systems

14.10 What performance measure(s) is (are) of greatest interest to each of the following? Explain your answers.

a. an interactive user

b. a batch-processing user

c. a designer of a real-time process control system

d. installation managers concerned with billing users for resource usage

e. installation managers concerned with projecting system loads for the next yearly budget cycle

f. installation managers concerned with predicting the performance improvements to be gained by adding

i. memory

ii. faster processors

iii. disk drives

14.11 There is a limit to how many measurements should be taken on any system. What considerations might cause you to avoid taking certain measurements?

14.12 Simulation is viewed by many as the most widely applicable performance evaluation technique.

a. Give several reasons for this.

b. Even though simulation is widely applicable, it is not as widely used as one might expect. Give several reasons why.

14.13 Some of the popular performance evaluation and monitoring techniques are

i. timings

ii. microbenchmarks

iii. synthetic programs

iv. vector-based methodology

v. simulations

vi. kernel programs

vii. hardware monitors

viii. analytic models

ix. software monitors

x. benchmarks

Indicate which of these techniques is best defined by each of the following. (Some items can have more than one answer.)

a. their validity might be jeopardized by making simplfying assumptions

b. weighted average of instruction timings

c. produced by skilled mathematicians

d. models that are run on a computer

e. useful for quick comparisons of "raw horsepower" for hardware

f. particularly valuable in complex software environments

g. a real program executed on a real machine

h. useful for determining the performance of operating system primitives

i. custom-designed programs to exercise specific features of a machine

j. a real program "executed on paper"

k. a production program

1. most commonly developed by using the techniques of queuing theory and Markov processes

m. often used when it is too costly or time consuming to develop a synthetic program

14.14 What performance evaluation techniques are most applicable in each of the following situations? Explain your answers.

a. An insurance company has a stable workload consisting of a large number of lengthy batch-processing production runs. Because of a merger, the company must increase its capacity by 50 percent. The company wishes to replace its equipment with a new computer system.

b. The insurance company described in (a) wishes to increase capacity by purchasing some additional memory and channels.

c. A computer company is designing a new, ultrahigh-speed computer system and wishes to evaluate several alternative designs.

d. A consulting firm that specializes in commercial data processing gets a large military contract requiring extensive mathematical calculations. The company wishes to determine if its existing computer equipment will process the anticipated load of mathematical calculations.

e. Management in charge of a multicomputer network needs to locate bottlenecks as soon as they develop and to reroute traffic accordingly.

f. A systems programmer suspects that one of the software modules is being called upon more frequently than originally anticipated. The programmer wants to confirm this before devoting substantial effort to recoding the module to make it execute more efficiently.

g. An operating systems vendor needs to test all aspects of the system before selling its product commercially.

14.15 On one computer system, the processor contains a BIPS meter that records how many billion instructions per second the processor is performing at any instant in time. The meter is calibrated from 0 to 4 BIPS in increments of 0.1 BIPS. All of the workstations to this computer are currently in use. Explain how each of the following situations might occur.

a. The meter reads 3.8 BIPS and the terminal users are experiencing good response times.

b. The meter reads 0.5 BIPS and the terminal users are experiencing good response times.

c. The meter reads 3.8 BIPS and the terminal users are experiencing poor response times.

d. The meter reads 0.5 BIPS and the terminal users are experiencing poor response times.

14.16 You are a member of a performance evaluation team working for a computer manufacturer. You have been given the task of developing a generalized synthetic program to facilitate the evaluation of a completely new computer system with an innovative instruction set.

a. Why might such a program be useful?

b. What features might you provide to make your program a truly general evaluation tool?

14.17 Distinguish between event-driven and script-driven simulators.

14.18 What does it mean to validate a simulation model?

How might you validate a simulation model of a small timesharing system (which already exists) with disk storage, several CRT terminals, and a shared laser printer?

14.19 What information might a performance evaluator get from an instruction execution trace?

a module execution trace?

Which of these is more useful for analyzing the operation of individual programs?

for analyzing the operation of systems?

14.20 How can bottlenecks be detected?

How can they be removed?

If a bottleneck is removed, should we expect a system's performance to improve? Explain.

14.21 What is a feedback loop?

Distinguish between negative and positive feedback.

Which of these contributes to system stability?

Which could cause instability? Why?

14.22 Workload characterization is an important part of any performance study. We must know what a computer is supposed to be doing before we can say much about how well it is doing it. What measurements might you take to help characterize the workload in each of the following systems?

a. a timesharing system designed to support program development

b. a batch-processing system used for preparing monthly bills for an electric utility with half a million customers

c. an advanced workstation used solely by one engineer

d. a microprocessor implanted in a person's chest to regulate heartbeat

e. a local area computer network that supports a heavily used electronic mail system within a large office complex

f. an air traffic control system for collision avoidance

g. a weather forecasting computer network that receives temperature, humidity, barometric pressure and other readings from 10,000 grid points throughout the country over communications lines.

h. a medical database management system that provides doctors around the world with answers to medical questions.

i. a traffic control computer network for monitoring and controlling the flow of traffic in a large city

14.23 A computer system manufacturer has a new multiprocessor under development. The system is modularly designed so that users may add new processors as needed, but the connections are expensive. The manufacturer must provide the connections with the original machine because they are too costly to install in the field. The manufacturer wants to determine the optimal number of processor connections to provide. The chief designer says that three is optimal. The designer believes that placing more than three processors on the system would not be worthwhile and that the contention for memory would be too great. What performance evaluation techniques would you recommend to help determine the optimal number of connections during the design stage of the project? Explain your answer.

14.24 Why are RISC programs generally longer than their CISC equivalents?

Given this, why might RISC programs execute faster than their CISC equivalents?

14.25 Compare branch prediction to branch predication.

Which is likely to yield higher performance? Why?

14.26 For each of the following features of modern processors, describe how this feature improves performance and indicate why its inclusion in a processor strays from either a pure RISC or pure CISC design.

a. superscalar architecture

b. out-of-order execution (OOO)

c. branch prediction

d. on-chip vector and floating point support

e. large instruction sets

f. decoding complex, multicycle instructions into simpler, single-cycle instructions.

14.27 How do EPIC processors differ from post-RISC processors?

Given these differences, what could be a hindrance to the adoption of EPIC processors?

Suggested Projects

14.28 It is often difficult to measure performance in a system without influencing the results. Prepare a research paper on the various ways benchmarks minimize the effect they have on a system's performance.

14.29 In recent years, graphics-rendering technology has improved at a phenomenal rate. Prepare a research paper on the architectural features that boost graphics card performance. Also discuss graphics card performance evaluation techniques.

14.30 Prepare a research paper on the IBM PowerPC 970 processor, which powers Apple's PowerMac G5 series. What type of instruction set does it have? What other technology improves the performance of this processor?

14.31 Prepare a research paper on tools such as *gprof to* allow application developers to profile their software.

14.32 Prepare a research paper surveying contemporary studies that compare RISC performance to CISC performance. Describe the strengths and weaknesses of each design philosophy in today's systems.

14.33 In the text, it was indicated that one weakness of the RISC approach is a dramatic increase in context-switching overhead. Give a detailed explanation of why this is so. Write a paper on context switching. Discuss the various approaches that have been used. Suggest how context switching might be handled efficiently in RISC-based systems.

Suggested Simulations

14.34 Obtain versions of popular benchmarks or synthetic programs discussed in the text, such as SYSMark or a SPECmark. Run these on several computers and prepare a comparison of the results. Are your results similar to those published

by the vendors? What factors might cause differences between the results? Describe your experience using these benchmarks and synthetic programs.

14.35 In this problem, you will undertake a reasonably detailed simulation study. You will write an event-driven simulation program using random-number generation to produce events probabilistically.

At one large batch-processing computer installation the management wants to decide what memory placement strategy will yield the best possible performance. The installation runs a computer with a large main memory that uses variable-partition multiprogramming. Each user program runs in a single group of contiguous storage locations. Users state their memory requirement in advance, and the operating system allocates each user the requested memory when the user's job is initiated. A total of 1024MB of main memory is available for user programs.

The storage requirements of jobs at this installation are distributed as follows.

10MB—30 percent of the jobs
20MB - 2 0 percent of the jobs
30MB—25 percent of the jobs
40MB —15 percent of the jobs
50MB —10 percent of the jobs

Execution times of jobs at this installation are independent of the jobs' storage requirements and are distributed as follows.

1 minute—30 percent of the jobs
2 minutes—20 percent of the jobs
5 minutes—20 percent of the jobs
10 minutes—10 percent of the jobs
30 minutes—10 percent of the jobs
60 minutes—10 percent of the jobs

The load on the system is such that there is always at least one job waiting to be initiated. Jobs are processed strictly first-come-first-served.

Write a simulation program to help you decide which memory placement strategy should be used at this installation. Your program should use random-number generation to produce the memory requirement and execution time for each job according to the distributions above. Investigate the performance of the installation over an eight-hour period by measuring throughput, storage utilization, and other items of interest to you for each of the following memory placement strategies.

a. first fit
b. best fit
c. worst fit

14.36 At the installation described in the previous problem, management suspects that the first-come-first-served job scheduling might not be optimal. In particular, they are concerned that longer jobs tend to keep shorter jobs waiting. A study of waiting jobs indicates that there are always at least 10 jobs waiting to be initiated (i.e., when 10 jobs are waiting and one is initiated, another arrives immediately). Modify your simulation program from the previous exercise so that job scheduling is now performed on a shortest-job-first basis. How does this affect performance for each of the memory placement strategies? What problems of shortest-job-first scheduling become apparent?

Recommended Reading

Lucas's 1971 paper, "Performance Evaluation and Monitoring," provides a survey of performance evaluation.[109] A discussion of recent trends in performance evaluation, particularly benchmarking, is found in *The Seventh Workshop on Hot Topics in Operating Systems* in the section entitled "Lies, Damn Lies, and Benchmarks."[110, 111] The SPEC Web site (www.specbench.org) and the *PC Magazine* benchmarks from VeriTest Web site (www.veritest.com/benchmarks/default.asp)[112] provide information on commercial benchmarks. Patterson and Hennesesy's *Computer Organization and Design* describes processor architecture.[113] Stokes[114] and Aletan[115] survey the histories and philosophies of RISC and CISC processors. Flynn, Mitchell and Mulder discuss some benefits of the CISC approach.[116] Schlansker and Rau explain the EPIC design philosophy.[117] The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Mogul, J., "Brittle Metrics in Operating System Research," *Proceedings of the Seventh Workshop on Hot Operating System Topics,* March 1999, pp. 90-95.
2. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.
3. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.
4. Ferrari, D.; G. Serazzi; and A. Zeigner, *Measurement and Tuning of Computer Systems,* Englewood Cliffs, NJ: Prentice Hall, 1983.
5. Anderson, G., "The Coordinated Use of Five Performance Evaluation Methodologies," *Communications of the ACM,* Vol 27, No. 2, February 1984, pp. 119-125.

6. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 105-106.
7. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 105-106.
8. Spinellis, D., "Trace: A Tool for Logging Operating System Call Transactions," *ACM SIGOPS Operating System Review,* October 1994, pp. 56-63.
9. Bull, J. M., and D. O'Neill, "A Microbenchmark Suite for OpenMP 2.0," *ACM SIGARCH Computer Architecture Notes,* Vol. 29, No. 5, December 2001, pp. 41-48.
10. Keckler, S., et al., "Exploiting Fine-Grain Thread Level Parallelism on the MIT Multi-ALU Processor," *Proceedings of the 25th Annual International Symposium on Computer Architecture,* October 1998, pp. 306-317.
11. Brown, A., and M. Seltzer, "Operating System Benchmarking in the Wake of Lmbench: A Case Study on the Performance of NetBSD on the Intel x86 Architecture," *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems* June 1997, pp. 214-224.
12. McVoy, L., and C. Staelin, "lmbench: Portable Tools for Performance Analysis," in *Proceedings of the 1996 USENIX Annual Technical Conference,* January 1996, pp. 279-294.
13. Brown, A., and M. Seltzer, "Operating System Benchmarking in the Wake of lmbench: A Case Study on the Performance of NetBSD on the Intel x86 Architecture," *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* June 1997; pp. 214-224.
14. McVoy, L., and Staelin, C, "lmbench: Portable tools for Performance Analysis," *In Proceedings of the 1996 USENIX Annual Technical Conference,* January 1996, pp. 279-294.
15. Brown, A., and M. Seltzer, "Operating System Benchmarking in the Wake of Lmbench: A Case Study on the Performance of NetBSD on the Intel x86 Architecture," *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* June 1997, pp. 214-224.
16. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 105-106.
17. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 105-106.
18. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 102-107.
19. Lucas, H., "Performance Evaluation and Monitoring," ACM *Computing Surveys,Vol.* 3, No. 3, September 1971, pp. 79-91.
20. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,Vol.* 3, No. 3, September 1971, pp. 79-91.
21. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.
22. Svobodova, L., *Computer Performance Measurement and Evaluation Methods: Analysis and Applications,* New York: Elsevier, 1977.
23. Kobayashi, H., *Modeling and Analysis: An Introduction to System Performance Evaluation Methodology,* Reading, MA: Addison-Wesley, 1978.
24. Lazowska, E., "The Benchmarking, Tuning, and Analytic Modeling of VAX/VMS," *Conference on Simulation, Measurement and Modeling of Computer Systems,* August 1979, pp. 57-64.
25. Sauer, C, and K. Chandy, *Computer Systems Performance Modeling,* Englewood Cliffs, NJ: Prentice Hall, 1981.
26. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.
27. Hindin, H., and M. Bloom, "Balancing Benchmarks Against Manufacturers' Claims," *UNIX World,* January 1988, pp. 42-50.
28. Mogul, J., "Brittle Metrics in Operating System Research," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 90-95.
29. "SpecWeb99," *SPEC (Standard Performance Evaluation Corporation),* September 26,2003, <www.specbench.org/web99/>.
30. Mogul, J., "Brittle Metrics in Operating System Research," *Proceedings of the Seventh Workshop on Hot Operating System Topics,* March 1999, pp. 90-95.
31. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 102-107.
32. "BAPCo Benchmark Products," *BAPCo,* August 5, 2002, <www.bapco.com/products.htm>.
33. "Overview of the TPC Benchmark C," *Transaction Processing Performance Council,* August 12, 2003, <www.tpc.org/tpcc/detail.asp>.
34. "SAP Standard Application Benchmarks," August 12, 2003, <www.sap.com/benchmark>.
35. Bucholz, W., "A Synthetic Job for Measuring System Performance," *IBM Journal of Research and Development,* Vol. 8, No. 4,1969, pp. 309-308.
36. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.
37. Weicker, R., "Dhrystone: A Synthetic Systems Programming Benchmark," *Communications of the ACM,* Vol. 21, No. 10, October 1984, pp. 1013-1030.
38. Dronek, G., "The Standards of System Efficiency," *Unix Review,* March 1985, pp. 26-30.
39. Wilson, P., "Floating-Point Survival Kit," *Byte,* Vol. 21, No. 3, March 1988, pp. 217-226.
40. "WinBench," *PC Magazine,* September 2, 2003, <www.etestinglabs.com/benchmarks/winbench/winbench.asp>.
41. "IOStone: A Synthetic File System Benchmark," *ACM SIGARCH Computer Architecture News,* Vol. 18, No. 2, June 1990, pp. 45-52.
42. "Hartstone: Synthetic Benchmark Requirements for Hard Real-Time Applications," *Proceedings of the Working Group on ADA Performance Issues,* 1990, pp. 126-136.
43. "Unix Technical Response Benchmark Info Page," May 17,2002, <www.unix.ualberta.ca/Benchmarks/benchmarks.html>.

44- Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.

45. Nance, R., "The Time and State Relationships in Simulation Modeling," *Communications of the ACM,* Vol. 24, No. 4, April 1981, pp. 173-179.

46. Keller, R., and F. Lin, "Simulated Performance of a Reduction-Based Multiprocessor," *Computer,* Vol. 17, No. 7, July 1984, pp. 70-82.

47. Overstreet, C, and R, Nance, "A Specification Language to Assist in Analysis of Discrete Event Simulation Models," *Communications of the ACM,* Vol. 28, No. 2, February 1985, pp. 190-201.

48. Jefferson, D., et al., "Distributed Simulation and the Time Warp Operating System," *Proceeding of the 11th Symposium on Operating System Principles,Vo*\. 21, No. 5, November 1987, pp. 77-93.

49. Overstreet, C, and R. Nance, "A Specification Language to Assist in Analysis of Discrete Event Simulation Models," *Communications of the ACM,* Vol. 28, No. 2, February 1985, pp. 190-201.

50. Gibson, J. et al., "FLASH vs. (Simulated) FLASH: Closing the Simulation Loop," *ACM SIGPLAN Notices,* Vol. 35, No. 11, November 2000, pp. 52-58.

51. Lucas, H, "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.

52. Plattner, B., and J. Nivergelt, "Monitoring Program Execution: A Survey," *Computer,Vol* 14,No. 11, November 1981,pp. 76-92.

53. Irving, R.; C. Higgins; and F. Safayeni, "Computerized Performance Monitoring Systems: Use and Abuse," *Computer Technology Form No. SA23-1057,*1986.

54. "Windows 2000 Performance Tools: Leverage Native Tools for Performance Monitoring and Tuning," *InformIT,* January 15,2001, <www.informit.com/content/index.asp?product_id=%7BA085E192-E708-4AAB-999E-5C339560EAA6%7D>.

55. Gavin, D, "Performance Monitoring Tools for Linux," *Linux Journal,* No. 56, December 1, 1998, **<www.linuxjournal.com/article.php?sid=2396>.**

56. "IA-32 Intel Architecture Software Developer's Manual," Vol. 3, *System Programmer's Guide,* 2002.

57. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.

58. Ferrari, D.; G Serazzi; and A. Zeigner, *Measurement and Tuning of Computer Systems,* Englewood Cliffs, NJ: Prentice Hall, 1983.

59. Bonomi, M., "Avoiding Coprocessor Bottlenecks," *Byte,* Vol. 13, No. 3, March 1988, pp. 197-204.

60. Coutois, P., "Decomposability, Instability, and Saturation in Multiprogramming Systems," *Communications of the ACM,* Vol. 18, No. 7,1975, pp. 371-376.

61. Hennessy, J., and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann Publishers, Inc., 1998, p. G-7.

62. Stokes, J., "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechmca.com/cpu/4q99/risc-cisc/rvc-1.html>.

63. Stokes, J., "Ars Technica: RISC vs. CISC in the Post RISC Era." October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-l.html>.

64. Daily, S., "Killer Hardware for Windows NT," *Windows IT Library,* January 1998 <www.windowsitlibrary.com/Content/435/02/toc.html>.

65. DeMone, P., "RISC vs. CISC Still Matters," February 13, 2000. <ctas.east.asu.edu/bgannod/CET520/Spring02/Projects/demone.htm>.

66. Hennessy, J. and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann Publishers, Inc., 1998, p. 525.

67. Rosen, S., "Electronic Computers—A Historical Survey," *ACM ComputingSurveys,Vol.* l,No. 1, January 1969, pp. 26-28.

68. Ramamoorthy, C. V., and H. F Li, "Pipeline Architecture," *ACM Computing Survey*'.s, Vol. 9, No. 1, January 1977, pp. 62-64.

69. Aletan, S., "An Overview of RISC Architecture," *Proceedings of the 1992 ACM/SIGAPP Symposium on Applied Computing: Technological Challenges of the 1990's,* 1992, pp. 11-12.

70. Hopkins, M., "Compiling for the RT PC ROMP," *IBM RT Personal Computer Technology,* 1986, pp. 81.

71. Coulter, N. S., and N. H. Kelly, "Computer Instruction Set Usage by Programmers: An Empirical Investigation," *Communications of the ACM,* Vol. 29, No. 7, July 1986, pp. 643-647.

72. Serlin, O., "MIPS, Dhrystones, and Other Tales," *Datamation,* Vol. 32, No. 11, June 1,1986, pp. 112-118.

73. Aletan, S., "An Overview of RISC Architecture," *Proceedings of the 1992 ACM/SIGAPP Symposium on Applied Computing: Technological Challenges of the 1990's,* 1992, p. 13.

74. Patterson, D. A., "Reduced Instruction Set Computers," *Communications of the ACM,Vol.* 28, No. 1, January 1985, pp. 8-21.

75. Hopkins, M., "Compiling for the RT PC ROMP," *IBM RT Personal Computer Technology,* 1986, pp. 81.

76. Patterson, D. A., "Reduced Instruction Set Computers," *Communications oftheACM,Vol.* 28, No. 1, January 1985, pp. 8-21.

77. Patterson, D., "Reduced Instruction Set Computers," *Communications oftheACM,Vol* 28, No. 1, January 1985,pp. 8-21.

78. Lilja, D.,"Reducing the Branch Penalty in Pipelined Processors." *Computer,Vol.* 21, No. 7, July 1988, pp. 47-55.

79. Huang, I., "Co-synthesis of Pipelined Structures and Structured Reordering Constraints for Instruction Set Processors," *ACM Transactions on Design Automation of Electronic Systems,* Vol. 6. No. 1, January 2001, pp. 93-121.

80. Lilja, D., "Reducing the Branch Penalty in Pipelined Processors," *Computer,Vol.* 21, No. 7, July 1988, pp. 47-55.

81. Wilson, R., "RISC Chips Explore Parallelism for Boost in Speed," *Computer Design,* January 1,1989, pp. 58-73.

82. Davidson, J. W., and R. A. Vaughan, "The Effect of Instruction Set Complexity on Program Size and Memory Performance," *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systerns,* 1987, pp. 60-64.

S3. Wilson, R., "RISC Chips Explore Parallelism for Boost in Speed," *Computer Design,* January 1,1989, pp. 58-73.

84. "IA-32 Intel Architecture Software Developer's Manual," Vol. 1: Basic Architecture, 2002.

85. Brehob, M., et al., "Beyond RISC-The Post-RISC Architecture," *Michigan State University Department of Computer Science, Technical Report CPS-96-11,* March 1996.

86. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-l.html>.

87. Stokes, J., "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-l.html>.

88. Brehob, M., et al., "Beyond RISC-The Post-RISC Architecture," *Michigan State University Department of Computer Science, Technical Report CPS-96-11,* March 1996.

89. DeMone, P., "RISC vs. CISC Still Matters," February 13, 2000, <ctas.east.asu.edu/bgannod/CET520/Spring02/Projects/demone.htm>.

90. Hennessy, J., and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann Publishers, Inc., 1998, p. 510.

91. Jouppi, N., "Superscalar vs. Superpipelined Machines," *ACM SIGARCH Computer Architecture News,* Vol. 16, No. 3, June 1988, pp. 71-80.

92. Ray, X; X Hoe; and B. Falsafi, "Dual Use of Superscalar Datapath for Transient-Fault Detection and Recovery," *Proceedings of the 34th Annual ACM/IEEE International Symposium on Microarchitecture,* December 2001, pp. 214-224.

93. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

94. Hwu, W. W., and Y. Patt, "Checkpoint Repair for Out-of-Order Execution Machines," *Proceedings of the 14th International Symposium on Computer Architecture,* June 1987, pp. 18-26.

95. Chuang, W., and B. Calder, "Predicate Prediction for Efficient Out-of-Order Execution," *Proceedings of 17th Annual International Conference on Supercomputing,* June 2003, pp. 183-192.

96. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

97. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

98. Young, C, and M. Smith, "Static Correlation Branch Prediction," *ACM Transactions on Programming Languages and Systems,* Vol. 21, No. 5, September 1999, pp. 1028-1075.

99. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

100. Stokes, J., "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

101. "Altivec Fact Sheet," *Motorola Corporation,* 2002, <e-www.motorola.com/files/32bit/doc/fact_sheet/ALTIVECCLANCE.pdf>.

102. Stokes, J., "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

103. Schlansker, M., and B. Rau, "EPIC: Explicitly Parallel Instruction Computing," *Computer,* Vol. 33, No. 2, February 2000, pp. 37-38.

104. Lowney, P., et al., "The Multiflow Trace Scheduling Compiler," *Journal of Supercomputing,* Vol. 7, May 1993:51-142.

105. Hennessy, J., and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaulmann Publishers, Inc., 1998, p. 528.

106. Schlansker, M., and B. Rau, "EPIC: Explicitly Parallel Instruction Computing," *Computer,* Vol. 33, No. 2, February 2000, pp. 38-39.

107. Schlansker, M., and B. Rau, "EPIC: Explicitly Parallel Instruction Computing," *Computer,* Vol. 33, No. 2, February 2000, pp. 41-44.

108. Schlansker, M., and B. Rau, "EPIC: Explicitly Parallel Instruction Computing," *Computer,* Vol. 33, No. 2, February 2000, pp. 41-44.

109. Lucas, H., "Performance Evaluation and Monitoring," *ACM Computing Surveys,* Vol. 3, No. 3, September 1971, pp. 79-91.

no. Mogul, X, "Brittle Metrics in Operating System Research," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 90-95.

in. Seltzer, M., et al., "The Case for Application-Specific Benchmarking," *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems,* March 1999, pp. 105-106.

112. "PC Magazine Benchmarks from VeriTest," *PC Magazine* <www.veritest.com/benchmarks/default.asp?visitor=>.

113. Hennessy, J., and D. Patterson, *Computer Organization and Design,* San Francisco, CA: Morgan Kaufmann Publishers, Inc., 1998, p. 528.

114. Stokes, X, "Ars Technica: RISC vs. CISC in the Post RISC Era," October 1999, <arstechnica.com/cpu/4q99/risc-cisc/rvc-1.html>.

115. Aletan, S., "An Overview of RISC Architecture," *Proceedings of the 1992 ACM/SIGAPP Symposium on Applied Computing: Technological Challenges of the 1990's,* 1992, pp. 11-12.

116. Flynn, M. X; C. L. Mitchell; and X M. Mulder, "And Now a Case for More Complex Instruction Sets," *Computer,* Vol. 20, No. 9, September 1987, pp. 71-83.

117. Schlansker, M., and Rau, B., "EPIC: Explicitly Parallel Instruction Computing," *Computer,* Vol. 33, No. 2, February 2000, pp. 41-44.

What's going to happen in the next decade is that
we'll figure out how to make parallelism work.

-David Kuck quoted in *TIME,* March 28, 1988-

"The question is," said Humpty Dumpty, "whish is to be master - that's all."

— Lewis Carroll —

I hope to see my Pilot face to face When I have crossed the bar.

— Alfred, Lord Tennyson —

"If seven maids with seven mops
Swept it for half a year,
Do you suppose," the Walrus said,
"That they could get it clear?"

—Lewis Carroll—

The most general definition of beauty ... Multeity in Unity.

—Samuel Taylor Coleridge—

Chapter 15

Multiprocessor Management

Objectives

After reading this chapter, you should understand:

- *multiprocessor architectures and operating system organizations.*
- *multiprocessor memory architectures.*
- *design issues specific to multiprocessor environments.*
- *algorithms for multiprocessor scheduling.*
- *process migration in multiprocessor systems.*
- *load balancing in multiprocessor systems.*
- *mutual exclusion techniques for multiprocessor systems.*

Chapter Outline

15.9 Multiprocessor Mutual Exclusion

15.1 Introduction

For decades, Moore's Law has successfully predicted an exponential rise in processor transistor count and performance, yielding ever more powerful processors. Despite these performance gains, researchers, developers, businesses and consumers continue to demand substantially more computing power than one processor can provide. As a result, **multiprocessing systems**—computers that contain more than one processor—are employed in many computing environments.

Large engineering and scientific applications that execute on supercomputers increase throughput by processing data in parallel on multiple processors. Businesses and scientific institutions use multiprocessing systems to increase system performance, scale resource usage to application requirements and provide a high degree of data reliability.[1] For example, the Earth Simulator in Yokohama, Japan—the most powerful supercomputer as of June, 2003—contains 5,f20 processors, each operating at 500MHz (see the Mini Case Study, Supercomputers). The system can

Mini Case Study

Supercomputers

Supercomputer is simply a term for the most powerful contemporary computers. The early supercomputers would be no match for today's inexpensive PCs. The speed of a supercomputer is measured in Flops (Floating-point operations per second).

The first supercomputer was the CDC 6600, manufactured by Control Data Corporation and designed by Seymour Cray, who became known as the father of supercomputing (see the Biographical Note, Seymour Cray).[2, 3] This machine, released in the early 1960's, processed 3 million instructions per second.[4, 5] It was also the first computer to use the RISC (reduced instruction set computing) architecture.[6,7] Roughly 10 years and several machines later, Cray designed the Cray-1, one of the earliest vector-processor models.[8] (Vector processors are discussed in Section 15.2.1, Classifying Sequential and Parallel Architectures.) The supercomputers that Cray designed dominated the high-performance field for many years.

According to the supercomputer tracking organization Top500 (www.top500.org), the fastest computer in the world (at the time this book was published) is NEC's Earth Simulator, located in Japan. It operates at a peak speed of roughly 35 teraflops, more than twice the speed of the second-fastest computer and tens of thousands of times faster than a typical desktop machine.[9, 10] The Earth Simulator consists of 5,120 vector processors grouped into 640 units of eight processors each, where each unit has 16GB of shared main memory, totalling 10TB of memory.[11, 12] These units are connected by 1500 miles of network cable and are linked to a 640TB disk system and a tape library that can hold over 1.5 petabytes (1,500,000,000,000,000 bytes).[13, 14] This enormous computing capacity is used for research in predicting environmental conditions and events.[15]

execute 35.86 Tflops (trillion floating point operations per second), enabling researchers to model weather patterns, which can be used to predict natural disasters and evaluate how human activities affect nature.[16, 17]

Multiprocessing systems must adapt to different workloads. In particular, the operating system should ensure that

- all processors remain busy
- processes are evenly distributed throughout the system
- execution of related processes is synchronized
- processors operate on consistent copies of data stored in shared memory
- mutual exclusion is enforced.

Techniques used to solve deadlock in multiprocessing and distributed systems are similar and are discussed in Chapter 17, Introduction to Distributed Systems.

This chapter describes multiprocessor architectures and techniques for optimizing multiprocessing systems. These techniques focus on improving performance,

Biographical Note

Seymour Cray

Seymour Cray is known as the father of supercomputing. Shortly after getting his Masters in Applied Mathematics from the University of Minnesota in 1951, he joined Engineering Research Associates (ERA), one of the first companies to build digital computers.[18, 19, 21] At ERA Cray designed the 1103, the first computer designed for scientific research that also sold well to the general market.[20, 22]

In 1957, Cray left the company with William Norris (the founder of ERA) to start Control Data Corporation.[2324-25] By 1960 Cray had designed the CDC 1604, the first computer that completely replaced vacuum tubes with transistors.[26, 27, 28] In 1962 Cray designed the CDC 6600, the first true "supercomputer," followed by the CDC 7600.[29, 30, 31] He always strove for simplicity, so the CDC 6600 featured a simplified architecture based on a smaller set of instructions—a design philosophy later called RISC (reduced instruction set computing).[32, 33, 34]

Seymour Cray founded Cray Research in 1972 to continue building large scientific computers.[35] The Cray-1, one of the earliest vector-processor computers, was released in 1976 (see Section 15.2.1, Classifying Sequential and Parallel Architectures).[36] The Cray-1 was much smaller yet more powerful than the CDC 7600.[37] It was also visually interesting, curving around to form a cylinder with a gap, and equipped with a bench running around the outside.[38] It ran at 133 megaflops, which was extraordinarily fast at that time, but slower than today's typical desktop PCs.[39] The Cray-1 was followed by three more Cray models, each progressively faster and more compact than the last. The computational unit (not including the memory banks and cooling equipment) of the Cray-4 was smaller than the human brain.[40]

fairness, cost and fault tolerance. Often, improvement in one of these parameter-occurs at the expense of the others. We also consider how design decisions can affect multiprocessor performance.

Self Review

1. Why are multiprocessors useful?
2. How do the responsibilities of a multiprocessor operating system differ from those of a uniprocessor system?

Ans: **1)** Many computer users demand more processing power than one processor can provide For example, businesses can use multiprocessors to increase performance and scale resource usage to application needs. **2)** The operating system must balance the workload of various processors, enforce mutual exclusion in a system where multiple processes can execute truly simultaneously and ensure that all processors are made aware of modifications to shared memory.

15.2 Multiprocessor Architecture

The term "multiprocessing system" encompasses any system containing more than one processor. Examples of multiprocessors include dual-processor personal computers, powerful servers that contain many processors and distributed groups of workstations that work together to perform tasks.

Throughout this chapter we present several ways to classify multiprocessors. In this section, we categorize multiprocessing systems by their physical properties, such as the nature of the system's datapath (i.e., the portion of the processor that performs operations on data), the processor interconnection scheme and how processors share resources.

15.2.1 Classifyeing Sequential and Parallel Architectures

Flynn developed an early scheme for classifying computers into increasingly parallel configurations. The scheme consists of four categories based on the different types of **streams** used by processors.[41] A stream is simply a sequence of bytes that is fed to a processor. A processor accepts two streams—an instruction stream and a data stream.

Single-instruction-stream, single-data-stream (SISD) computers are the simplest type. These are traditional uniprocessors in which a single processor fetches one instruction at a time and executes it on a single data item. Techniques such as pipelining, very long instruction word (VLIW) and superscalar design can introduce parallelism into SISD computers. Pipelining divides an instruction's execution path into discrete stages. This allows the processor to process multiple instructions simultaneously, as long as at most one instruction occupies each stage during a clock cycle. VLIW and superscalar techniques simultaneously issue multiple independent instructions (from one instruction stream) that execute in different execution units. VLIW relies on the compiler to determine which instructions to issue at any given clock cycle, whereas superscalar design requires that the processor make this decision.[42] Additionally, Intel's Hyper-Threading technology introduces parallelism by creating two virtual processors from one physical processor. This gives a multiprocessor-

enabled operating system the impression that it is running on two processors that each execute at a little less than half the speed of the physical processor.[43]

Multiple-instruction-stream, single-data-stream (MISD) computers are not commonly used. A MISD architecture would have several processing units that act on a single stream of data. Each unit would execute a different instruction on the data and pass the result to the next unit.[44]

Single-instruction-stream, multiple-data-stream (SIMD) computers issue instructions that act on multiple data items. A SIMD computer consists of one or more processing units. A processor executes a SIMD instruction by performing the same instruction on a block of data (e.g., adding one to every element in an array). If there are more data elements than processing units, the processing units fetch additional data elements for the next cycle. This can increase performance relative to SISD architectures, which would require a loop to perform the same operation ome data element at a time. A loop contains many conditional tests, requires the SISD processor to decode the same instruction multiple times and requires the SISD processor to read data one word at a time. By contrast, SIMD architectures read in a block of data at once, reducing costly memory-to-register transfers. SIMD architectures are most effective in environments in which a system applies the same instruction to large data sets.[45, 46]

Vector processors and **array processors** use a SIMD architecture. A vector processor contains one processing unit that executes each vector instruction on a set of data, performing the same operation on each data element. Vector processors rely on extremely deep pipelines (i.e., ones containing many stages) and high clock speeds. Deep pipelines permit the processor to perform work on several instructions at a time so that many data elements can be manipulated at once. An array processor contains several processing units that perform the same instruction in parallel on many data elements. Array processors (often referred to as **massively parallel processors)** might contain tens of thousands of processing elements. Therefore, array processors are most efficient when manipulating large data sets. Vector and array processing are useful in scientific computing and graphics manipulation where the same operation must be applied to a large data set (e.g., matrix transformations).[47, 48] The Connection Machine (CM) systems built by Thinking Machines, Inc., are examples of array processors.[49]

Multiple-instruction-stream, multiple-data-stream (MIMD) computers are multiprocessors in which the processing units are completely independent and operate on separate instruction streams.[50] However, these systems typically contain hardware that allows processors to synchronize with each other when necessary, such as when accessing a shared peripheral device.

Self Review

1. Thread-level parallelism (TLP) refers to the execution of multiple independent threads in parallel. Which multiprocessor architecture exploits TLP?
2. (T/F) Only SIMD processor architectures exploit parallelism.

Ans: **1)** Only MIMD and MISD architectures can execute multiple threads at once. However, the threads executed by a MISD computer manipulate the same data and are not independent. Therefore, only MIMD systems can truly exploit TLP. **2)** False. SISD systems use techniques such as pipelines, VLIW and superscalar design to exploit parallelism; MISD processors execute multiple threads at once; and MIMD processors exploit TLP as described in the previous answer.

15.2.2 Processor Interconnection Schemes

The **interconnection scheme** of a multiprocessor system describes how the system's components, such as processors and memory modules, are physically connected. The interconnection scheme is a key issue for multiprocessor designers because it affects the system's performance, reliability and cost. An interconnection scheme consists of **nodes** and **links.** Nodes are composed of system components and/or **switches** that route messages between components. A link is a connection between two nodes. In many systems, a single node might contain one or more processors, their associated caches, a memory module and a switch. In large-scale multiprocessors, we sometimes abstract the concept of a node and denote a group of nodes as a single supernode.

Designers use several parameters to evaluate interconnection schemes. A node's **degree** is the number of nodes to which it is connected. Designers try to minimize a node's degree to reduce the complexity and cost of its communication interface. Nodes with larger degrees require more complex communication hardware to support communication between the node and its neighbor nodes (i.e., nodes connected to it).[51]

One technique for measuring an interconnection scheme's fault tolerance is to count the number of communication links that must fail before the network is unable to function properly. This can be quantified using the **bisection width—**the minimum number of links that need to be severed to divide the network into two unconnected halves. Systems with larger bisection widths are more fault tolerant than those with smaller bisection widths because more components must fail before the entire system fails.

An interconnection scheme's performance largely depends on communication latency between nodes. This can be measured in several ways, one of which is the average latency. Another performance measure is the **network diameter—**the shortest distance between the two most remote nodes in the interconnection scheme. To determine the network diameter, consider all pairs of nodes in the network and find the shortest path length for each pair—computed by totaling the number of links traversed—then find the largest of these paths. A small network diameter indicates low communication latency and higher performance. Finally, system architects attempt to minimize the **cost of an interconnection scheme,** which is equal to the total number of links in a network.[52]

In the following subsections, we enumerate several successful interconnection models and evaluate them based on the preceding criteria. Many real systems implement variations of these models. For example, they might add extra communi-

cation links to increase fault tolerance (by increasing the bisection width) and performance (by decreasing the network diameter).

Shared Bus

The **shared bus** network organization uses a single communication path (i.e., route through which messages travel) between all processors and memory modules (Fig. 15.1).[53] The components' bus interfaces handle transfer operations. The bus is passive, and the components arbitrate among themselves to use the bus. Only one transfer can take place at a time on the bus, because the bus cannot convey two electrical signals at once. Therefore, before a component initiates a transfer, it must first check that both the bus and the destination component are available. One problem with shared buses—**contention**—arises when several components wish to use the bus at once. To reduce contention and bus traffic, each processor maintains its own local cache as shown in Fig. 15.1. When the system can fulfill a memory request from a processor's cache, the processor does not need to communicate over the bus with a memory module. Another option **is** to construct a **multiple shared bus architecture,** which reduces contention by providing multiple buses that service communication requests. However, these require complex bus arbitration logic and additional links, which increases the system's cost.[54]-[55]

The shared bus is a simple and inexpensive scheme to connect a small number of processors. New components can be added to the system by attaching them to the bus, and software handles the detection and identification of the bus components. However, due to contention for the single communication path, shared bus organizations do not scale beyond a small number of processors (in practice 16 or 32 is the maximum).[56] Contention is exacerbated by the fact that processor speed has increased faster than bus bandwidth. As processors get faster, it takes fewer processors to saturate a bus.

Shared buses are dynamic networks, because communication links are formed and discarded (through the shared bus) during execution. Therefore, the criteria used to evaluate interconnection schemes that we discussed earlier do not apply; these criteria are based on static links, which do not change during execution. How-

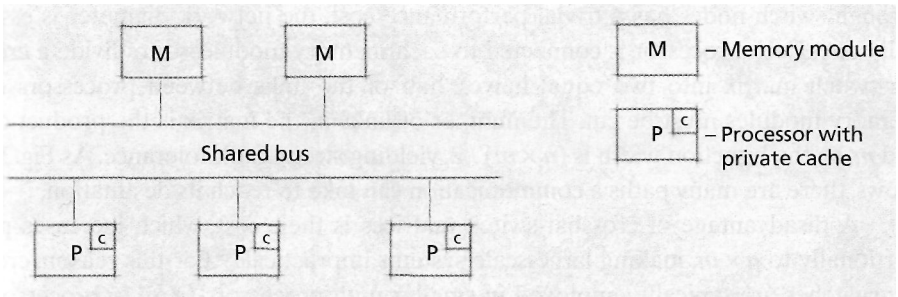

Figure 15.1 | *Shared bus multiprocessor organization.*

ever, compared to other interconnection schemes, a shared bus with several processors is fast and inexpensive, but not particularly fault tolerant—if the shared bus fails, the components cannot communicate.[57]

Designers can leverage the benefits of shared buses in multiprocessors with larger numbers of processors. In these systems, maintaining a single shared bus (or several buses) that connects all processors is impractical, because the bus becomes saturated easily. However, designers can divide the system's resources (e.g., processors and memory) into several small supernodes. The resources within a supernode communicate via a shared bus, and the supernodes are connected using one of the more scalable interconnection schemes described in the sections that follow. Such systems attempt to keep most communication traffic within a supernode to exploit the fast bus architecture, while enabling communication between supernodes.[58] Most multiprocessors with a small number of processors, such as dual-processor Intel Pentium systems, use a shared bus architecture.[59]

Crossbar-Switch Matrix

A crossbar-switch matrix provides a separate path from every processor to even memory module (Fig. 15.2).[60] For example, if there are *n* processors and *m* memory modules, there will be *n* x *m* total switches that connect each processor to each memory module. We can imagine the processors as the rows of a matrix and the memory modules as the columns. In larger networks, nodes often consist of processor and memory components. This improves memory-access performance (for those accesses between a processor and its associated memory module). In the case of a crossbar-switch matrix, this reduces the interconnection scheme's cost. In this design, each node connects to a switch of degree of *p* - 1, where *p* is the number of processor-memory nodes in the system (i.e., in this case, *m* - *n* because each node contains the same number of processors and memory modules).[61, 62]

A crossbar-switch matrix can support data transmissions to all nodes at once, but each node can accept at most one message at a time. Contrast this with the shared bus, which supports only one transmission at a time. A switch uses an arbitration algorithm such as "service the requesting processor that has been serviced least recently at this switch" to resolve multiple requests. The crossbar-switch design provides high performance. Because all nodes are linked to all other nodes and transmission through switch nodes has a trivial performance cost, the network diameter is essentially one. Each processor is connected to each memory module, so to divide a crossbar-switch matrix into two equal halves, half of the links between processors and memory modules must be cut. The number of links in the matrix is the product of *n* and *m,* so the bisection width is *(n* x *m) / 2,* yielding strong fault tolerance. As Fig. 15.2 shows, there are many paths a communication can take to reach its destination.

A disadvantage of crossbar-switch matrices is their cost, which increases proportionally *to nxm,* making large-scale systems impractical.[63] For this reason, crossbar-switches are typically employed in smaller multiprocessors (e.g., 16 processors). However, as hardware costs decline, they are being used more frequently in larger systems. The Sun UltraSPARC-Ill uses crossbar switches to share memory.[64]

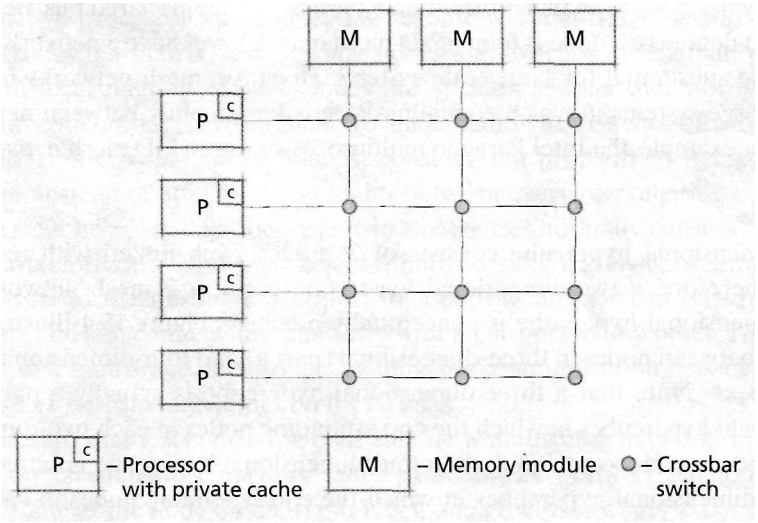

Figure 15.2 | *Crossbar-switch matrix multiprocessor organization.*

2-D Mesh Network

In a **2-D mesh network** interconnection scheme, each node consists of one or more processors and a memory module. In the simplest case (Fig. 15.3), the nodes in a mesh network are arranged in a rectangle of n rows and m columns, and each node is connected with the nodes directly north, south, east and west of it. This is called a **4-connected 2-D mesh network.** This design keeps each node's degree small, regardless of the number of processors in a system—the corner nodes have a degree of two, the edge nodes have a degree of three and the interior nodes have a degree of four. In Fig. 15.3, where $n = 4$ and $m = 5$, the 2-D mesh network can be divided into two equal halves by cutting the five links between the second and third rows of nodes. In fact, if n is even and m is odd the bisection width is $m + 1$ if $m > n$ and is n otherwise. If the 2-D mesh contains an even number of rows and columns, the bisection width is the

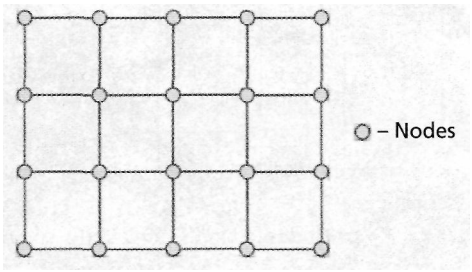

Figure 15.3 | *4-connected 2-D mesh Network.*

smaller of *m* and *n*. Although not as fault tolerant as a crossbar-switch matrix, a 2-D mesh network is more so than other simple designs, such as a shared bus. Because the maximum degree of a node is four, a 2-D mesh network will have a network diameter that is too substantial for large-scale systems. However, mesh networks have been used in large systems in which communication is kept mainly between neighboring nodes. For example, the Intel Paragon multiprocessor uses a 2-D mesh network.[65]

Hypercube

An n-dimensional **hypercube** consists of 2^n nodes, each linked with *n* neighbor nodes. Therefore, a two-dimensional hypercube is a 2 x 2 mesh network, and a three-dimensional hypercube is conceptually a cube.[66] Figure 15.4 illustrates connections between nodes in three-dimensional (part a) and four-dimensional (part b) hypercubes.[67] Note that a three-dimensional hypercube is actually a pair of two-dimensional hypercubes in which the corresponding nodes in each two-dimensional hypercube are connected. Similarly, a four-dimensional hypercube is actually a pair of three-dimensional hypercubes in which the corresponding nodes in each three-dimensional hypercube are connected.

A hypercube's performance scales better than that of a 2-D mesh network, because each node is connected by *n* links to other nodes. This reduces the network diameter relative to a 2-D mesh network. For example, consider a 16-node multiprocessor implemented as either a 4 x 4 mesh network or a 4-dimensional hypercube. A 4 x 4 mesh network has a network diameter of 6, whereas a 4-dimensional hypercube has a network diameter of 4. In some hypercubes, designers add communication links between nonneighbor nodes to further reduce the network diameter.[68] A hypercube's fault tolerance also compares favorably with that of other designs. However, the increased number of links per node increases a hypercube's cost relative to that of a mesh network.[69]

The hypercube interconnection scheme is efficient for connecting a modest number of processors and is more economical than a crossbar-switch matrix. The nCUBE system used in streaming media and digital-advertising systems employs hypercubes of up to 13 dimensions (8,192 nodes).[70]

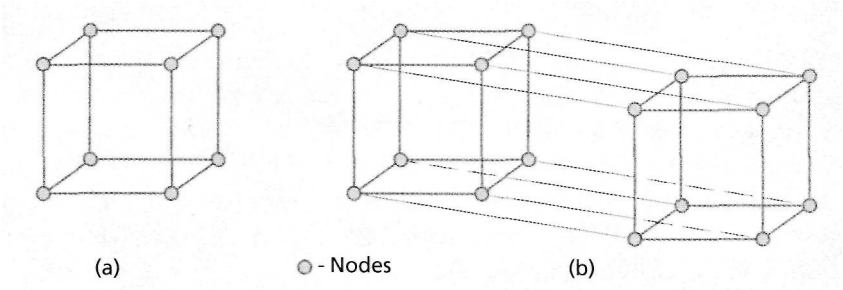

Figure 15.4 | *3- and 4-dimensional hypercubes.*

Multistage Networks

An alternative processor interconnection scheme is a **multistage network.**[71] As in the crossbar-switch matrix design, some nodes are switches rather than processor nodes with local memory. Switch nodes are smaller, simpler and can be packed more tightly, increasing performance. To understand the benefits of a multistage network over a crossbar-switch matrix, consider the problem of flying between small cities. Instead of offering direct flights between every pair of cities, airlines use large cities as "hubs." A flight between two small cities normally consists of several "legs" in which the traveller first flies to a hub, possibly travels between hubs and finally travels to the destination airport. In this way, airlines can schedule fewer total flights and still connect any small city with an airport to any other. The switching nodes in a multistage network act as hubs for communication between processors, just as airports in large cities do for airlines.

There are many schemes for constructing a multistage network. Figure 15.5 shows a popular multistage network called a **baseline network.**[72] Each nodes on the left is the same as the node on the right. When one processor wants to communicate with another, the message travels through a series of switches. The leftmost switch corresponds to the least significant (rightmost) bit of the destination processor's ID; the middle switch corresponds to the middle bit; and the rightmost switch corresponds to the most significant (leftmost) bit.

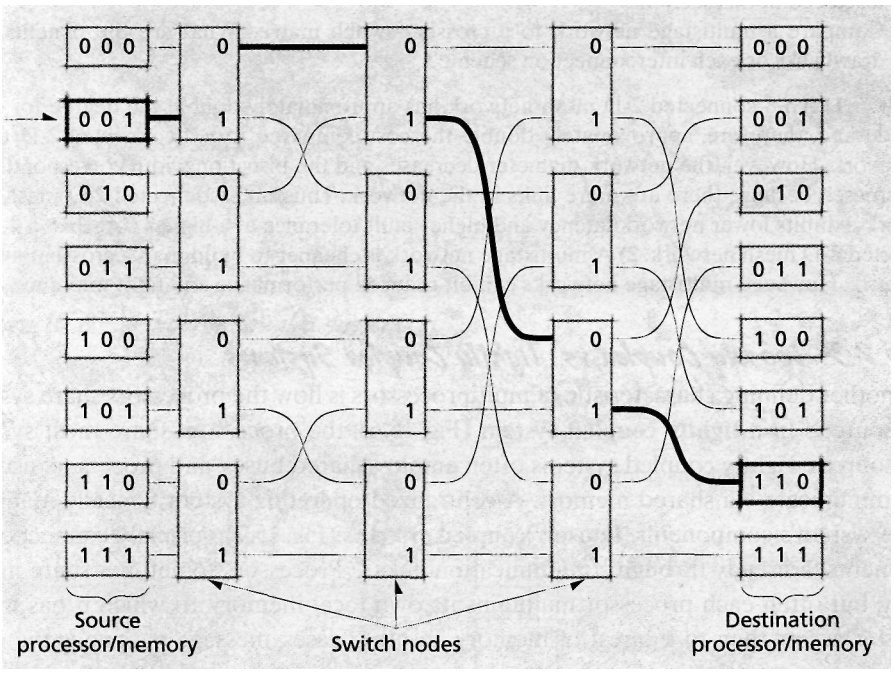

Figure 15.5 | *Multistage baseline network.*

For example, consider the bold path in Fig. 15.5 from the processor with ID 001 to the processor with ID 110. Begin by following the path from processor 001 to the switch. Next, determine the least significant bit (i.e., 0) in the destination processor's ID (110) and follow that path from the current switch to the next switch. From the second switch, follow the path that corresponds to the middle bit (i.e.. 1) in the destination processor's ID. Finally, from the third switch, follow the path that corresponds to the most significant bit (i.e., 1) in the destination processor's ID.

Multistage networks represent a cost-performance compromise. This design employs simple hardware to connect large numbers of processors. Any processor can communicate with any other without routing the message through intermediate processors. However, a multistage network has a larger network diameter, so communication is slower than with a crossbar-switch matrix—each message must pass through multiple switches. Also, contention can develop at switching elements, which might degrade performance. IBM's SP series multiprocessor, which evolved to the POWER4, uses a multistage network to connect its processors. The IBM ASCI-White, which can perform over 100 trillion operations per second, is based on the POWER3-II multiprocessor.[73, 74]

Self Review

1. An 8-connected 2-D mesh network includes links to diagonal nodes as well as the links in Fig. 15.3. Compare 4-connected and 8-connected 2-D mesh networks using the criteria discussed in this section.
2. Compare a multistage network to a crossbar-switch matrix. What are the benefits and drawbacks or each interconnection scheme?

Ans: **1)** An 8-connected 2-D mesh network has approximately double the degree for each node and, therefore, approximately double the cost compared to a 4-connected 2-D mesh network. However, the network diameter decreases and the bisection width correspondingly increases, because there are more links in the network. Thus, an 8-connected 2-D mesh network exhibits lower network latency and higher fault tolerance at a higher cost than a 4-connected 2-D mesh network. **2)** A multistage network is cheaper to build than a crossbar-switch matrix. However, multistage networks exhibit reduced performance and fault tolerance.

15.2.3 Loosely Coupled vs. Tightly Coupled Systems

Another defining characteristic of multiprocessors is how the processors share system resources. In a **tightly coupled system** (Fig. 15.6), the processors share most system resources. Tightly coupled systems often employ shared buses, and processors usually communicate via shared memory. A centralized operating system typically manages the system's components. **Loosely coupled systems** (Fig. 15.7) normally connect components indirectly through communication links.[75] Processors sometimes share memory, but often each processor maintains its own local memory, to which it has much faster access than to the rest of memory. In other cases, message passing is the only form of communication between processors, and memory is not shared.

In general, loosely coupled systems are more flexible and scalable than tightly coupled systems. When components are loosely connected, designers can easily add

components to or remove components from the system. Loose coupling also increases fault tolerance because components can operate independently of one another. However, loosely coupled systems typically are less efficient, because they communicate by passing messages over a communications link, which is slower than communicating through shared memory. This also places a burden on operating systems programmers, who typically hide most of the complexity of message passing from application programmers. Japan's Earth Simulator is an example of a loosely coupled system.[76]

By contrast, tightly coupled systems perform better but are less flexible. Such systems do not scale well, because contention for shared resources builds up quickly as processors are added. For this reason, most systems with a large number of processors are loosely coupled. In a tightly coupled system, designers can optimize interactions between components to increase system performance. However, this decreases the system's flexibility and fault tolerance, because one component relies on other components. A dual-processor Intel Pentium system is an example of a tightly coupled system.[77]

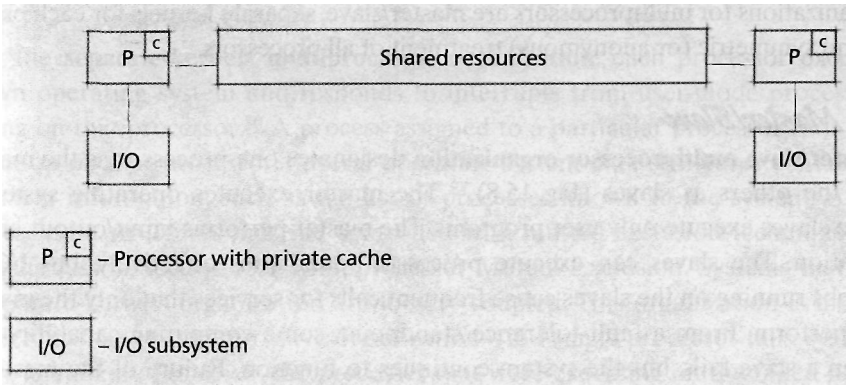

Figure 15.6 | *Tightly coupled system*

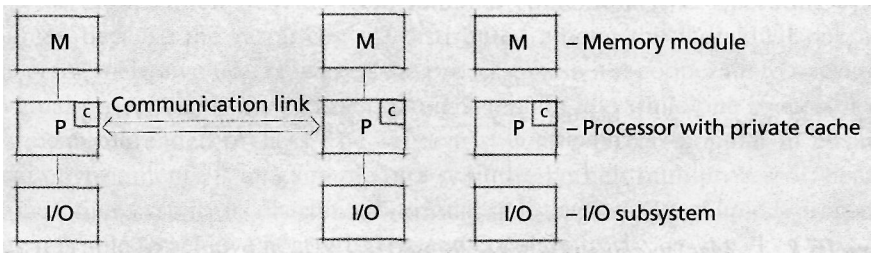

Figure 15.7 | *Loosely coupled system.*

Self Review

1. Why are many small multiprocessors constructed as tightly coupled systems? Why are many large-scale systems loosely coupled?
2. Some systems consist of several tightly coupled groups of components connected together in a loosely coupled fashion. Discuss some motivations for this scheme.

Ans: **1)** A small system would typically be tightly coupled because there is little contention for shared resources. Therefore, designers can optimize the interactions between system components, thus providing high performance. Typically, large-scale systems are loosely coupled to eliminate contention for shared resources and reduce the likelihood that one failed component will cause systemwide failure. **2)** This scheme leverages the benefits of both interconnection organizations. Each group is small, so tightly coupled systems provide the best performance within a group. Making the overall system loosely coupled reduces contention and increases the system's flexibility and fault tolerance.

15.3 Multiprocessor Operating System Organizations

In organization and structure, multiprocessor operating systems differ significantly from uniprocessor operating systems. In this section, we categorize multiprocessors based on how they share operating system responsibilities. The basic operating system organizations for multiprocessors are master/slave, separate kernels for each processor and symmetric (or anonymous) treatment of all processors.

15.3.1 Master/Slave

The **master/slave multiprocessor organization** designates one processor as the master and the others as slaves (Fig. 15.8).[78] The master executes operating system code; the slaves execute only user programs. The master performs input/output and computation. The slaves can execute processor-bound jobs effectively, but I/O-bound jobs running on the slaves cause frequent calls for services that only the master can perform. From a fault-tolerance standpoint, some computing capability is lost when a slave fails, but the system continues to function. Failure of the master processor is catastrophic and halts the system. Master/slave systems are tightly coupled because all slave processors depend on the master.

The primary problem with master/slave multiprocessing is the hardware asymmetry, because only the master processor execute the operating system. When

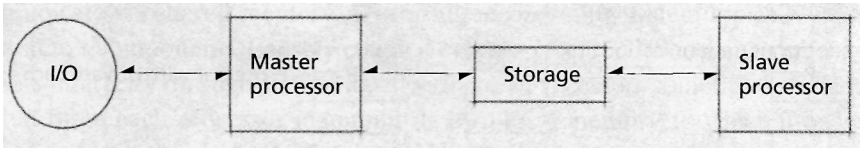

Figure 15.8 | *Master/Slave multiprocessing.*

a process executing on a slave processor requires the operating system's attention, the slave processor generates an interrupt and waits for the master to handle the interrupt.

The nCUBE hypercube system is an example of a master/slave system. Its many slave processors handle computationally intensive tasks associated with manipulating graphics and sound.[79]

Self Review

1. (T/F) Master/slave multiprocessors scale well to large-scale systems.
2. For what type of environments are master/slave multiprocessors best suited?

Ans **1)** False. Only one processor can execute the operating system. Contention will build between processes that are awaiting services (such as I/O) that only the master processor can provide. **2)** Master/slave processors are best suited for environments that mostly execute processor-bound processes. The slave processors can execute these processes without intervention from the master processor.

15.3.2 Separate Kernels

In the **separate-kernels multiprocessor organization,** each processor executes its own operating system and responds to interrupts from user-mode processes running on that processor.[80] A process assigned to a particular processor runs to completion on that processor. Several operating system data structures contain global system information, such as the list of processes known to the system. Access to these data structures must be controlled with mutual exclusion techniques, which we discuss in Section 15.9, Multiprocessor Mutual Exclusion. Systems that use the separate-kernels organization are loosely coupled. This organization is more fault tolerant than the master/slave organization—if a single processor fails, system failure is unlikely. However, the processes that were executing on the failed processor cannot execute until they are restarted on another processor.

In the separate-kernels organization, each processor controls its own dedicated resources, such as files and I/O devices. I/O interrupts return directly to the processors that initiate those interrupts.

This organization benefits from minimal contention over operating system resources, because the resources are distributed among the individual operating systems for their own use. However, the processors do not cooperate to execute an individual process, so some processors might remain idle while one processor executes a multithreaded process. The Tandem system, which is popular in business-critical environments, is an example of a separate-kernels multiprocessor; because the operating system is distributed throughout the loosely coupled processing nodes, it is able to achieve nearly 100 percent availability.[81]

Self Review

1. Why is the separate-kernels organization more fault tolerant than the master/slave organization?
2. For what type of environment would separate-kernels multiprocessor be useful?

Ans: **1)** The separate-kernels organization is loosely coupled. Each processor has its own resources and does not interact with other processors to complete its tasks. If one processor fails, the remainder continue to function. In master/slave organizations, if the master fails none of the slave processors can perform operating system tasks that must be handled by the master. **2)** A separate-kernels multiprocessor is useful in environments where processes do not interact, such as a cluster of workstations in which users execute independent programs.

15.3.3 Symmetrical Organization

Symmetrical multiprocessor organization is the most complex organization to implement, but is also the most powerful.[82]-[83] *[Note:* This organization should not be confused with symmetric multiprocessor (SMP) systems, which we discuss in Section 15.4.1.] The operating system manages a pool of identical processors, any one of which can control any I/O device or reference any storage unit. The symmetry makes it possible to balance the workload more precisely than with the other organizations.

Because many processors might be executing the operating system at once, mutual exclusion must be enforced whenever the operating system modifies shared data structures. Hardware and software conflict-resolution techniques are important.

Symmetrical multiprocessor organizations are generally the most fault tolerant. When a processor fails, the operating system removes that processor from its pool of available processors. The system degrades gracefully while repairs are made (see the Operating Systems Thinking feature, Graceful Degradation). Also, a process running on a symmetrical system organization can be dispatched to any processor. Therefore, a process does not rely on a specific processor as in the separate-kernels organization. The operating system "floats" from one processor to the next.

One drawback of the symmetrical multiprocessor organization is contention for operating system resources, such as shared data structures. Careful design of system data structures is essential to prevent excessive locking that prevents the operating system from executing on multiple processors at once. A technique that minimizes contention is to divide the system data structures into separate and independent entities that can be locked individually.

Even in completely symmetrical multiprocessing systems, adding new processors does not cause system throughput to increase by the new processors' rated capacities. There are many reasons for this, including additional operating system overhead, increased contention for system resources and hardware delays in switching and routing transmissions between an increased number of components. Several performance studies have been performed; one early BBN Butterfly system with 256 processors operated 180 to 230 times faster than a single-processor system.[84]

Self Review

1. Why would doubling the number of processors in a symmetrical organization not double the total processing power?
2. What are some benefits of the symmetrical organization over the master/slave and separate-kernels organizations?

Ans: **1)** Adding processors increases contention for resources and increases operating system overhead, offsetting some of the performance gains attained from adding processors.
2) The symmetrical organization is more scalable than master/slave because all processors can execute the operating system; this also makes the system more fault tolerant. Symmetrical organization provides better cooperation between processors than separate kernels. This facilitates IPC and enables systems to exploit parallelism better.

15.4 Memory Access Architectures

So far, we have classified multiprocessor systems by hardware characteristics and by how processors share operating system responsibilities. We can also classify multiprocessor systems by how they share memory. For example, consider a system with tew processors and a small amount of memory. If the system contains a group of memory modules that is easily accessible by all processors (e.g., via a shared bus), the system can maintain fast memory access. However, systems with many processors and memory modules will saturate the bus that provides access to these memory modules. In this case, a portion of memory can be closely tied to a processor so

Operating Systems Thinking

Graceful Degradation

Things do go wrong. Individual components of systems do fail. Entire systems fail. The question is, "What should a system do after experiencing some degree of failure?" Many systems are designed to degrade gracefully (i.e., continue operating after a failure, but at reduced levels of service). A classic example of graceful degradation occurs in a symmetric multiprocessing system in which any processor can run any process. If one of the processors fails, the system can still function with the remaining processors, but with reduced performance. Graceful degradation occurs in many disk systems that, upon detecting a failed portion of the disk, simply "map around" the failed areas, enabling the user to continue storing and retrieving information on that disk. When a router fails on the Internet, the Internet continues to handle new transmissions by sending them to the remaining functioning routers. Providing for graceful degradation is an important task for operating systems designers, especially as people grow increasingly reliant on hardware devices that eventually fail.

that the processor can access its memory more efficiently. Designers, therefore, must weigh concerns about a system's performance, cost and scalability when determining the system's memory-access architecture. The following sections describe several common memory architectures for multiprocessor systems.

15.4.1 Uniform Memory Access

Uniform-memory-access (LIMA) multiprocessor architectures require all processors to share the system's main memory (Fig. 15.9). This is a straightforward extension of a uniprocessor memory architecture, but with multiple processors and memory modules. Typically, each processor maintains its own cache to reduce bus contention and increase performance. Memory-access time is uniform for any processor accessing any data item, except when the data item is stored in a processor's cache or there is contention on the bus. UMA systems are also called **symmetric multiprocessor (SMP)** systems because any processor can be assigned any task and all processors share all resources (including memory, I/O devices and processes). UMA multiprocessors with a small number of processors typically use a shared bus or a crossbar-switch matrix interconnection network. I/O devices are attached directly to the interconnection network and are equally accessible to all processors.[85]

UMA architectures are normally found in small multiprocessor systems (typically two to eight processors). UMA multiprocessors do not scale well—a bus quickly becomes saturated when more than a few processors access main memery simultaneously, and crossbar-switch matrices become expensive even for modest-sized systems.[86,87]

Self Review

1. Why are mesh networks and hypercubes inappropriate interconnection schemes for UMA systems?
2. How is a UMA system "symmetric?"

Ans: **1)** Mesh networks and hypercubes place processors and memory at each node, so local memory can be accessed faster than remote memory; thus memory-access times are not uni-

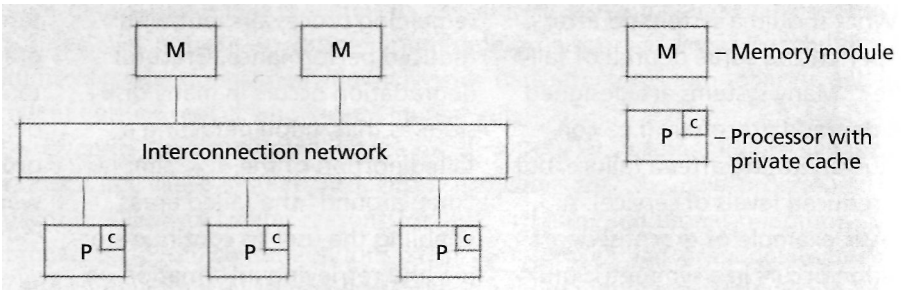

Figure 15.9 | *UMA multiprocessor.*

form. **2)** It is symmetric because any processor can be assigned any task and all processors share all resources (including memory, I/O devices and processes).

15.4.2 Nonuniform Memory Access

Nonuniform-memory-access (NUMA) multiprocessor architectures address UMA's scalability problems. The primary bottleneck in a large-scale UMA system is access to shared memory—performance degrades due to contention from numerous processors attempting to access shared memory. If a crossbar-switch matrix is used, the interconnection scheme's cost might increase substantially to facilitate multiple paths to shared memory. NUMA multiprocessors handle these problems by relaxing the constraint that memory-access time should be uniform for all processors accessing any data item.

NUMA multiprocessors maintain a global shared memory that is accessible by all processors. The global memory is partitioned into modules, and each node uses one of these memory modules as the processor's local memory. In Fig. 15.10, each node contains one processor, but this is not a requirement. Although the interconnection scheme's implementation might vary, processors are connected directly to their local memory modules and connected indirectly (i.e., through one of the interconnection schemes discussed in Section 15.2.2) to the rest of global memory. This arrangement provides faster access to local memory than to the rest of global memory, because access to global memory requires traversing the interconnection network.

The NUMA architecture is highly scalable because it reduces bus collisions when a processor's local memory services most of the processor's memory requests. A NUMA system can implement a strategy that moves pages to the processor on which those pages are accessed most frequently—a technique called page migration, which is discussed in detail in Section 15.5.2, Page Replication and Migration. Typically, NUMA systems can support a large number of processors, but they are more complex to design than UMAs, and systems with many processors can be expensive to implement.[88,89]

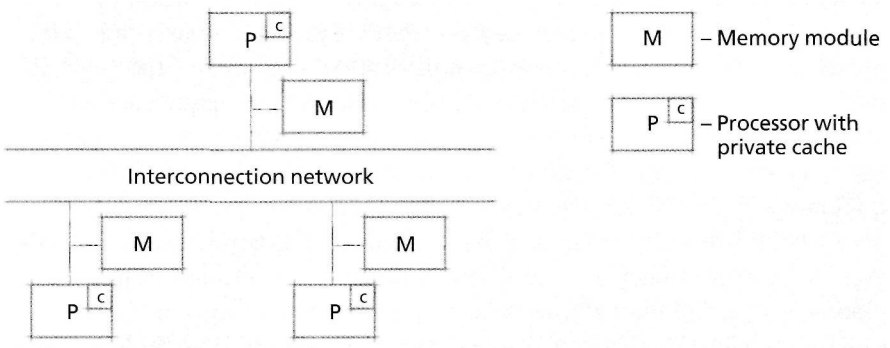

Figure 15.10 | *NUMA multiprocessor.*

Self Review

1. List some advantages of NUMA multiprocessors over UMA. List some disadvantages.
2. What issues does the NUMA design raise for programmers and for operating systems designers?

Ans: **1)** NUMA multiprocessors are more scalable than UMA multiprocessors because NUMA multiprocessors remove the memory-access bottleneck of UMA systems by using large numbers of processors. UMA multiprocessors are more efficient for small systems because there is little memory contention and access to all memory in a UMA system is fast. **2)** If two processes executing on processors at separate nodes use shared memory for IPC, then at least one of the two will not have the memory item in its local memory, which degrades performance. Operating system designers place processes and their associated memory together on the same node. This might require scheduling a process on the same processor each time the process executes. Also, the operating system should be able to move pages to different memory modules based on demand.

15.4.3 Cache-Only Memory Architecture

As described in the previous section, each node in a NUMA system maintains its own local memory, which processors from other nodes can access. Often, local memory access is dramatically faster than global memory access (i.e., access to another node's local memory). **Cache-miss latency**—the time required to retrieve data that is not in the cache—can be significant when the requested data is not present in local memory. One way to reduce cache-miss latency is to reduce the number of memory requests serviced by remote nodes. Recall that NUMA systems place data in the local memory of the processor that accesses the data most frequently. This is impractical for a compiler or programmer to implement, because data access patterns change dynamically. Operating systems can perform this task, but can move only page-size chunks of data, which can slow data migration. Also, different data items on a single page often are accessed by processors at different nodes.[90]

Cache-Only Memory Architecture (COMA) multiprocessors (also called Cache-Only Memory Access multiprocessors) use a slight variation of NUMA to address this memory placement issue (Fig. 15.11). COMA multiprocessors include one or more processors, each with its associated cache, and a portion of the global shared memory. However, the memory associated with each node is organized as a large cache known as an **attraction memory (AM).** This allows the hardware to migrate data efficiently at the granularity of a **memory line**—equivalent to a cache line, but in main memory, and typically four or eight bytes.[91] Also, because each processor's local memory is viewed as a cache, different AMs can have copies of the same memory line. With these design modifications, data often resides in the local memory of the processor that uses the data most frequently, which reduces the average cache-miss latency. The trade-offs are memory overhead from duplicating data items in multiple memory modules, and complicated hardware and protocols to ensure that memory updates are reflected in each processor's AM. This overhead results in higher latency for those cache misses that are serviced remotely.[92]

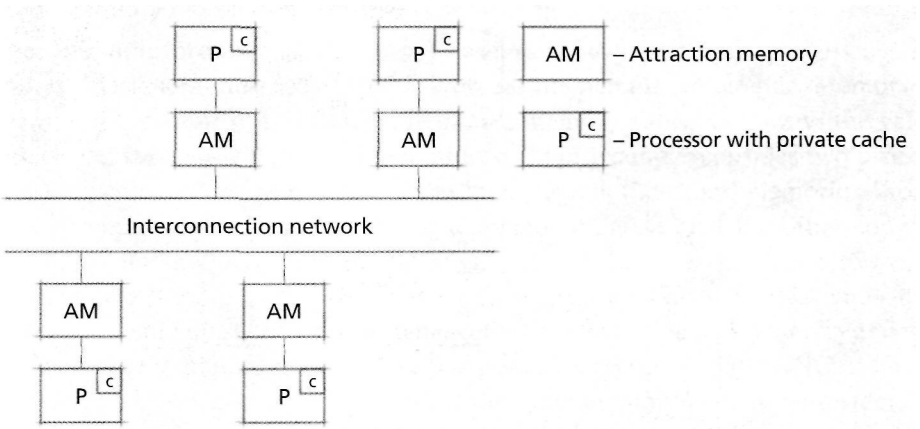

Figure 15.11 | *COMA multiprocessor.*

Self Review

1. What problems inherent in NUMA multiprocessors does the COMA design address?
2. (T/F) The COMA design always increases performance over a NUMA design.

Ans: **1)** Cache misses serviced at remote nodes in both NUMA and COMA systems typically require much more time than cache misses serviced by local memory. Unlike NUMA systems, COMA multiprocessors use hardware to move copies of data items to a processor's local memory (the AM) when referenced. **2)** False. COMA multiprocessors reduce the *number* of cache misses serviced remotely, but they also add overhead. In particular, cache-miss latency increases for cache misses that are serviced remotely, and synchronization is required for data that is copied into several processors' attraction memory.

15.4.4 No Remote Memory Access

UMA, NUMA and COMA multiprocessors are tightly coupled. Although NUMA (and COMA to a lesser extent) multiprocessors scale well, they require complex software and hardware. The software controls access to shared resources such as memory; the hardware implements the interconnection scheme. **NO-Remote-Memory-Access (NORMA) multiprocessors** are loosely coupled multiprocessors that do not provide any shared global memory (Fig. 15.12). Each node maintains its own local memory, and often, NORMA multiprocessors implement a common **shared virtual memory (SVM).** On an SVM system, when a process requests a page that is not in its processor's local memory, the operating system loads the page into local memory from another memory module (i.e., from a remote computer over a network) or from secondary storage (e.g., a disk).[93]-[94] Nodes in NORMA systems that do not support SVM must share data through explicit message passing. Google, which powers its service using 15,000 low-end servers located across the world, is an example of a distributed NORMA multiprocessor system.[95]

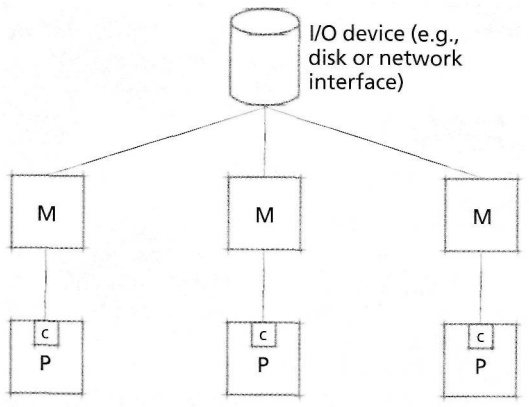

Figure 15.12 | *NORMA multiprocessor.*

NORMA systems are the simplest to build, because they do not require a complex interconnection scheme. However, the absence of shared global memory requires application programmers to implement IPC via message passing and remote procedure calls. On many systems, using shared virtual memory is inefficient, because the system would have to send entire data pages from one processor to the next, and these processors often are not in the same physical machine.[96] We discuss SVM in more detail in Section 15.5.3, Shared Virtual Memory.

Because NORMA multiprocessors are loosely coupled, it is relatively easy to remove or add nodes. NORMA multiprocessors are distributed systems governed by a single operating system, rather than networked computers each with its own operating system. If a node in a NORMA system fails, a user can simply switch to another node and continue working. If one percent of the nodes fails in a NORMA multiprocessor, the system merely runs one percent slower.[97]

Self Review

1. Why are NORMA multiprocessors ideal for workstation clusters?
2. In what environments are NORMA multiprocessors not useful?

Ans: **1)** Different users in a workstation cluster usually do not share memory. However, they share resources such as a file system and processing power, which NORMA systems provide. Also, it is relatively easy to add or remove nodes in NORMA multiprocessors, which can be useful for scaling the system's processing capabilities. **2)** NORMA multiprocessors are not useful in shared-memory environments, especially systems with a single user or a few processors. These can be implemented more efficiently with UMA or NUMA designs.

15.5 Multiprocessor Memory Sharing

When multiple processors with private caches or local memories access shared memory, designers must address the problem of **memory coherence.** Memory is coherent if the value obtained from reading a memory address is always the value that was most recently written to that address.[98] For example, consider two processors that maintain a separate copy of the same page in their local memories. If one processor modifies its local copy, the system should ensure that the other copy of the page is updated to reflect the latest changes. Multiprocessors that allow only one copy of a page to be resident in the system at a time (such as UMA and small NUMA systems) must still ensure **cache coherence**—reading a cache entry reflects the most recent update to that data. Large-scale systems (such as NORMA or large-scale NUMA systems) often permit the same memory page to be resident in the local memory of several processors. These systems must integrate memory coherence into their implementation of shared virtual memory.

Other important design considerations are page placement and replication. Local memory access is much faster than global memory access, so ensuring that data accessed by a processor resides in that processor's local memory can improve performance. There are two common strategies for addressing this issue. **Page replication** maintains multiple copies of a memory page, so that the page can be accessed quickly at multiple nodes (see the Operating Systems Thinking feature, Data Replication and Coherency). **Page migration** transfers pages to the node (or nodes, when used with page replication) where processors access a page most. In Section 15.5.2, Page Replication and Migration, we consider implementations of these two strategies.

Self Review

1. Why is memory coherence important?
2. What are the advantages of allowing multiple copies of the same page to exist in one system?

Operating Systems Thinking

Data Replication and Coherency

We will see several examples of data being duplicated for various reasons in operating systems. Sometimes this is done for backup to ensure recoverability if one copy of the data is lost. It is also done for performance where some copies of the data may be accessed faster than others. Often, it is crucial to ensure coherency, (i.e., that all copies of the data either are identical or will be made so before any differences could cause a problem).

Ans: **1)** Without memory coherence, there is no guarantee that a particular copy of data is the most up-to-date version. Data must be coherent to ensure that applications produce correct results. **2)** If two processes executing on different processors repeatedly read the same memory page, replicating the page allows both processors to keep the page in local memory so that it can be accessed quickly.

15.5.1 Cache Coherence

Memory coherence became a design consideration when caches appeared, because computer architectures allowed different access paths to data (i.e., through the cache copy or the main memory copy). In multiprocessor systems, coherence is complicated by the fact that each processor maintains a private cache.

UMA Cache Coherence

Implementing cache coherence protocols for UMA multiprocessors is simple because caches are relatively small and the bus connecting shared memory is relatively fast. When a processor updates a data item, the system must also update or discard all instances of that data in other processors' caches and in main memory.[99] This can be accomplished by **bus snooping** (also called **cache snooping).** In this protocol, a processor "snoops" the bus by determining whether a requested write from another processor is for a data item in the processor's cache. If the data resides in the processor's cache, the processor removes the data item from its cache. Bus snooping is simple to implement, but generates additional traffic on the shared bus. Alternatively, the system can maintain a centralized directory that records the items that reside in each cache and indicates when to remove stale data (i.e., data that does not reflect the most recent update) from a cache. Another option is for the system to allow only one processor to cache a particular memory item.[100]

Cache-Coherent NUMA (CC-NUMA)

Cache-coherent NUMAs (CC-NUMAs) are NUMA multiprocessors that impose cache coherence. In a typical CC-NUMA architecture, each physical memory address is associated with a **home node,** which is responsible for storing the data item with that main-memory address. (Often, the home node is simply determined by the higher-order bits in the address.) When a cache miss occurs at a node, that node contacts the home node associated with the requested memory address. If the data item is clean (i.e., no other node has a modified version of the data item in its cache), the home node forwards it to the requesting processor's cache. If the data item is dirty (i.e., another node has written to the data item since the last time the main memory entry was updated), the home node forwards the request to the node with the dirty copy; this node sends the data item to both the requestor and the home node. Similarly, requests to modify data are performed via the home node. The node that wishes to modify data at a particular memory address requests exclusive ownership of the data. The most recent version of the data (if not already in the modifying node's cache) is obtained in the same manner as a read request. After the

modification, the home node notifies other nodes with copies of the data that the data was modified.[101]

This protocol is relatively simple to implement, because all reads and writes first contact the home node. Although this might seem inefficient, this coherence protocol requires a maximum of only three network communications. (Consider how much traffic would be generated if a writing node had to contact all other nodes.) This protocol also makes it easy to distribute the load throughout the system—by assigning each node as the home node for approximately the same number of addresses—which increases fault tolerance and reduces contention. However, this protocol can perform poorly if most data accesses come from remote nodes.

Self Review

1. What are the advantages and disadvantages of bus snooping?
2. Why is it difficult for the system to establish a home node for memory that is referenced by several different nodes over time? How can the CC-NUMA home-based protocol be modified to support such behavior?

Ans: **1)** Bus snooping is easy to implement and it enables cache coherence. However, it generates additional bus traffic. **2)** If memory-access patterns for a memory region change frequently, then it is difficult to decide which node should be the home node for that region. Although one node might make the majority of the references to that item now, this will soon change and most of the references will come from a remote node. A remedy would be to dynamically change the home node of a data item (this, of course, would imply changing the item's physical memory address).

15.5.2 Page Replication and Migration

NUMA systems have higher memory-access latency than UMA multiprocessors, which limits NUMA's performance. Therefore, maximizing the number of cache misses serviced by local memory is an important NUMA design consideration.[102] The COMA design, described in Section 15.4.3, is one attempt to address the NUMA latency issue. CC-NUMA systems address the latency issue by implementing page-migration or page-replication strategies.[103]

Replicating a page is straightforward. The system copies all data from a remote page to a page in the requesting processor's local memory. To maintain memory coherence, the system uses a data structure that records where all duplicate pages exist. We discuss different strategies for memory coherence in the next section. Page migration occurs in a similar fashion, except that after a page has been replicated, the original node deletes the page from its memory and flushes any associated TLB or cache entries.[104]

Although migrating and replicating pages has obvious benefits, these strategies can degrade performance if they are not implemented correctly. For example, remotely referencing a page is faster than migrating or replicating that page. Therefore, if a process references a remote page only once, it would be more efficient not to migrate or replicate the page.[105] In addition, some pages are better candidates for replication than for migration, and vice versa. For example, pages that are read fre-

quently by processes at different processors would benefit from replication. by enabling all processes to access that page from local memory. Also, the drawback of replication—maintaining coherency on writes—would not be a problem for a read-only page. A page frequently modified by one process is a good candidate for migration; if other processes are reading from that page, replicating the page would not be viable because these processes would have to fetch updates continually from the writing node. Pages that are frequently written by processes at more than one processor are not good candidates for either replication or migration, because they would be migrated or replicated after each write operation.[106]

To help determine the best strategy, many systems maintain each page's access history information. A system might use this information to determine which pages are frequently accessed—these should be considered for replication or migration The overhead of replicating or migrating a page is not justified for pages that are not frequently accessed. A system also might maintain information on which remote processors are accessing the page and whether these processors are reading from, or writing to, the page. The more page-access information the system gathers, the better decision the system can make. However, gathering this history and transferring it during migration incurs overhead. Thus, the system must collect enough information to make good replication or migration decisions without incurring excessive overhead from maintaining this information.[107]

Self Review

1. How could an ineffective migration/replication strategy degrade performance?
2. For what types of pages is replication appropriate? When is migration appropriate?

Ans: **1)** Migration and replication are more costly than remotely referencing a page. They are beneficial only if the page will be referenced remotely multiple times from the same node. An ineffective strategy might migrate or replicate pages that are, for example, modified by many nodes, requiring frequent updates and thus reducing performance. Also, manipulating pages access history information imposes additional overhead that can degrade performance.
2) Pages that are read frequently by multiple processors (but not modified) should be replicated. Pages that are written by only one remote node should be migrated to that node.

15.5.3 Shared Virtual Memory

Sharing memory on small, tightly coupled multiprocessors, such as UMA multiprocessors, is a straightforward extension of uniprocessor shared memory because all processors access the same physical addresses with equal access latency. This strategy is impractical for large-scale NUMA multiprocessors due to remote-memory-access latency, and it is impossible for NORMA multiprocessors, which do not share physical memory. Because IPC through shared memory is easier to program than IPC via message passing, many systems enable processes to share memory located on different nodes (and perhaps in different physical address spaces) through shared virtual memory (SVM). SVM extends uniprocessor virtual memory concepts by ensuring memory coherence for pages accessed by various processors.[108]

Two issues facing SVM designers are selecting which coherence protocol to use and when to apply it. The two primary coherence protocols are invalidation and write broadcast. First we describe these protocols, then we consider how to implement them.

Invalidation

In the **invalidation** memory-coherence approach, only one processor may access a page while it is being modified—this is called page ownership. To obtain page ownership, a processor must first invalidate (deny access to) all other copies of the page. After the processor obtains ownership, the page's access mode is changed to read/write, and the processor copies the page into its local memory before accessing it. To obtain read access to a page, a processor must request that the processor with read/write access change its access mode to read-only access. If the request is granted, other processors can copy the page into local memory and read it. Typically, systems employ the policy of always granting requests unless a processor is waiting to acquire page ownership; in the case that a request is denied, the requestor must wait until the page is available. Note that multiple processes modifying a single page concurrently leads to poor performance, because competing processors repeatedly invalidate the other processors' copies of the page.[109]

Write Broadcast

In the **write broadcast** memory-coherence approach, the writing processor broadcasts each modification to the entire system. In one version of this technique, only one processor may obtain write ownership of a page. Rather than invalidating all other existing copies of the page, the processor updates all copies.[110] In a second version, several processors are granted write access to increase efficiency. Because this scheme does not require writers to obtain write ownership, processors must ensure that various updates to a page are applied in the correct order, which can incur significant overhead.[111]

Implementing Coherence Protocols

There are several ways to implement a memory-coherence protocol, though designers must attempt to limit memory-access bus traffic. Ensuring that every write is reflected at all nodes as soon as possible can decrease performance; however, weakening coherence constraints can produce erroneous results if programs use stale data. In general, an implementation must balance performance and data integrity.

Systems that use **sequential consistency** ensure that all writes are immediately reflected in all copies of a page. This scheme does not scale well to large systems due to communication cost and page size. Internode communication is slow compared to accessing local memory; if a process repeatedly updates the same page, many wasteful communications will result. Also, because the operating system manipulates pages, which often contain many data items, this strategy might result in **false sharing,** where processes executing at separate nodes might need access to unrelated data on the same page. In this case, sequential consistency effectively requires

the two processes to share the page, even though their modifications do not affect one another.[112]

Systems that use **relaxed consistency** perform coherence operations periodically. The premise behind this strategy is that delaying coherence for remote data by a few seconds is not noticeable to the user and can increase performance and reduce the false sharing effect.[113]

In **release consistency,** a series of accesses begin with an **acquire operation** and end with a **release operation.** All updates between the acquire and the release are batched as one update message after the release.[114] **In lazy release consistency,** the update is delayed until the next acquire attempt for the modified memory. This reduces network traffic, because it eliminates coherence operations for pages that are not accessed again.[115]

One interesting consequence of lazy release consistency is that only a node that attempts to access a modified page receives the modification. If this node also modifies the page, a third node would need to apply two updates before it can access the page. This can lead to substantial page-update overhead. One way to reduce this problem is to provide periodic global synchronization points at which all data must be consistent.[116] An alternative way to reduce this problem is to use a **home-based consistency** approach similar to that used for cache coherence in a CC-NUMA. The home node for each page is responsible for keeping an updated version of the page. Nodes that issue an acquire to a page that has been updated since that its last release must obtain the update from the home node.[117]

The **delayed consistency** approach sends update information when a release occurs, but nodes receiving updates do not apply them until the next acquisition. These updates can be collected until a new acquire attempt, at which point the node applies all update information. This does not reduce network traffic, but it improves performance because nodes do not have to perform coherence operations as frequently[118]

Lazy data propagation notifies other nodes that a page has been modified at release time. This notification does not supply the modified data, which can reduce bus traffic. Before a node acquires data for modification, it determines if that data has been modified by another node, If so, the former node retrieves update information from the latter. Lazy data propagation has the same benefits and drawbacks as lazy release consistency. This strategy significantly reduces communication traffic, but requires a mechanism to control the size of update histories, such as a global synchronization point.[119]

Self Review

1. What is a benefit of relaxed consistency? A drawback?
2. In many relaxed consistency implementations, data may not be coherent for several seconds. Is this is a problem in all environments? Give an example where it might be a problem.

Ans: **1)** Relaxed consistency reduces network traffic and increases performance. However, the system's memory can be incoherent for a significant period of time, which increases the likelihood that processors will operate on stale data. **2)** This is not a problem in all environments, considering that even sequential consistency schemes may use networks exhibiting latencies of a second or two. However, latency of this magnitude is not desirable for a supercomputer in which many processes interact and communicate through shared memory. The extra latency degrades performance due to frequent invalidations.

15.6 Multiprocessor Scheduling

Multiprocessor scheduling has goals similar to those of uniprocessor scheduling—the system attempts to maximize throughput and minimize response time for all processes. Further, the system must enforce scheduling priority

Unlike uniprocessor scheduling algorithms which only determine in what order processes are dispatched, multiprocessor scheduling algorithms also must determine to which processor they are dispatched, which increases scheduling complexity. For example, multiprocessor scheduling algorithms should ensure that processors are not idle while processes are waiting to execute.

When determining the processor on which a process is executed, the scheduler considers several factors. For example, some strategies focus on maximizing parallelism in a system to exploit application concurrency. Systems often group collaborating processes into a job. Executing a job's processes in parallel improves performance by enabling these processes to execute truly simultaneously. This section introduces several **timesharing scheduling** algorithms that attempt to exploit such parallelism by scheduling collaborative processes on different processors.[120] This enables the processes to synchronize their concurrent execution more effectively.

Other strategies focus on **processor affinity—**the relationship of a process to a particular processor and its local memory and cache. A process that exhibits high processor affinity executes on the same processor through most or all of its life cycle. The advantage is that the process will experience more cache hits and, in the case of a NUMA or NORMA design, possibly fewer remote page accesses than if it were to execute on several processors throughout its life cycle. Scheduling algorithms that attempt to schedule a process on the same processor throughout its life cycle maintain **soft affinity,** whereas algorithms that schedule a process on only one processor maintain **hard affinity.**[121]

Space-partitioning scheduling algorithms attempt to maximize processor affinity by scheduling collaborative processes on a single processor (or single set of processors) under the assumption that collaborative processes will access the same shared data, which is likely stored in the processor's caches and local memory.[122] Therefore, space-partitioning scheduling increases cache and local memory hits. However, it can limit throughput, because these processes typically do not execute simultaneously.[123]

Multiprocessor scheduling algorithms are generally classified as job blind or job aware. **Job-blind scheduling** policies incur minimal scheduling overhead,

because they do not attempt to enhance a job's parallelism or processor affinity. **Job-aware scheduling** evaluates each job's properties and attempts to maximize each job's parallelism or processor affinity, which increases performance at the cost of increased overhead.

Many multiprocessor scheduling algorithms organize processes in **global run queues.**[124] Each global run queue contains all the processes in the system that are ready to execute. Global run queues might be used to organize processes by priority, by job or by which process executed most recently.[125]

Alternatively, systems might use a **per-processor run queue.** This is typical of large, loosely coupled systems (such as NORMA systems) in which cache hits and references to local memory should be maximized. In this case, processes are associated with a specific processor and the system implements a scheduling policy for that processor. Some systems use **per-node run queues;** each node might contain more than one processor. This is appropriate for a system in which a process is tied to a particular group of processors. We describe the related issue of process migration, which entails moving processes from one per-processor or per-node run queue to another, in Section 15.7, Process Migration.

Self Review

1. What types of processes benefit from timesharing scheduling? From space-partitioning scheduling?
2. When are per-node run queues more appropriate than global run queues?

Ans: **1)** Timesharing scheduling executes related processes simultaneously, improving performance for processes that interact frequently because processes can react to messages or modifications to shared memory immediately. Space-partitioning scheduling is more appropriate for processes that must sequentialize access to shared memory (and other resources) because their processors are likely to have cached shared data, which improves performance. **2)** Per-node run queues are more appropriate than global run queues in loosely coupled systems, where processes execute much less efficiently when accessing remote memory.

15.6.1 Job-Blind Multiprocessor Scheduling

Job-blind multiprocessor scheduling algorithms schedule jobs or processes on any available processor. The three algorithms described in this section are examples of job-blind multiprocessor scheduling algorithms. In general, any uniprocessor scheduling algorithm, such as those described in Chapter 8, can be implemented as a job-blind scheduling multiprocessor algorithm.

First-Come-First-Served (FCFS) Process Scheduling

First-come-first-served (FCFS) process scheduling places arriving processes in a global run queue. When a processor becomes available, the scheduler dispatches the process at the head of the queue and runs it until the process relinquishes the processor.

FCFS treats all processes fairly by scheduling them according to their arrival times. However, FCFS might be considered unfair because long processes make

short processes wait, and low-priority processes can make high-priority processes wait—although a preemptive version of FCFS can prevent the latter from occurring. Typically, FCFS scheduling is not useful for interactive processes, because FCFS cannot guarantee short response times. However, FCFS is easy to implement and eliminates the possibility of indefinite postponement—once a process enters the queue, no other process will enter the queue ahead of it.[126]-[127]

Round-Robin Process (RRprocesss) Multiprocessor Scheduling

Round-robin process (RRprocess) scheduling places each *ready* process in a global run queue. RRprocess scheduling is similar to uniprocessor round-robin scheduling—a process executes for at most one quantum before the scheduler dispatches a new process for execution. The previously executing process is placed at the end of the global run queue. This algorithm prevents indefinite postponement, but does not facilitate a high degree of parallelism or processor affinity because it ignores relationships among processes.[128]-[129]

Shortest-Process-First (SPF) Multiprocessor Scheduling

A system can also implement the **shortest-process-first (SPF)** scheduling algorithm, which dispatches the process that requires the least amount of time to run to completion.[130] Both preemptive and nonpreemptive versions of SPF exhibit lower average waiting times for interactive processes than FCFS does, because interactive processes typically are "short" processes. However, a longer process can be indefinitely postponed if shorter processes continually arrive before it can obtain a processor. As with all job-blind algorithms, SPF does not consider parallelism or processor affinity.

Self Review

1. Is a UMA or a NUMA system more suitable for job-blind multiprocessor scheduling?
2. Which job-blind scheduling strategy discussed in this section is most appropriate for batch processing systems and why?

Ans: **1)** A UMA system is more appropriate for a job-blind algorithm. NUMA systems often benefit from scheduling algorithms that consider processor affinity, because memory access time depends on the node at which a process executes, and nodes in a NUMA system have their own local memory. **2)** SPF is most appropriate because it exhibits high throughput, an important goal for batch-processing systems.

15.6.2 Job-Aware Multiprocessor Scheduling

Although job-blind algorithms are easy to implement and incur minimal overhead, they do not consider performance issues specific to multiprocessor scheduling. For example, if two processes that communicate frequently do not execute simultaneously, they might spend a significant amount of their time busy waiting, which degrades overall system performance. Furthermore, in most multiprocessor systems,

each processor maintains its own private cache. Processes in the same job often access the same memory items, so scheduling one job's processes on the same processor tends to increase cache hits and improves memory-access performance. In general, job-aware process-scheduling algorithms attempt to maximize parallelism or processor affinity, at the cost of greater scheduling algorithm complexity.

Smallest-Number-of-Processes-First (SNPF) Scheduling

The **smallest-number-of-processes-first (SNPF)** scheduling algorithm, which can be either preemptive or nonpreemptive, uses a global job priority queue. A job's priority is inversely proportional to the number of processes in the job. If jobs containing the same number of processes compete for a processor, the job that has waited the longest receives priority. In nonpreemptive SNPF scheduling, when a processor becomes available, the scheduler selects a process from the job at the head of the queue and allows it to execute to completion. In preemptive SNPF scheduling, if a new job arrives with fewer processes, it receives priority and its processes are dispatched immediately[131]-[132] SNPF algorithms improve parallelism, because processes that are associated with the same job can often execute concurrently. However, the SNPF algorithms do not attempt to improve processor affinity. Further, it is possible for jobs with many processes to be postponed indefinitely.

Round-Robin Job (RRJob) Scheduling

Round-robin job (RRJob) scheduling employs a global job queue from which each job is assigned to a group of processors (although not necessarily the same group each time the job is scheduled). Every job maintains its own process queue. If the system contains p processors and uses a quantum of length q, then a job receives a total of $p \times q$ of processor time when dispatched. Typically, a job does not contain exactly p processes that each exhaust one quantum (e.g., a process may block before its quantum expires). Therefore, RRJob uses round-robin scheduling to dispatch the job's processes until the job consumes the entire $p \times q$ quanta, the job completes or all of the job's processes block. The algorithm also can divide the $p \times q$ quanta equally among the processes in the job, allowing each to execute until it exhausts its quantum, completes or blocks. Alternatively, if a job has more than p processes, it can select p processes to execute for a quantum length of q.[133]

Similar to RRprocess scheduling, this algorithm prevents indefinite postponement. Further, because processes from the same job execute concurrently, this algorithm promotes parallelism. However, the additional context-switching overhead of round-robin scheduling can reduce job throughput.[134]-[135]

Cischeduling

Coscheduling (or **gang scheduling)** algorithms employ a global run queue that is accessed in round-robin fashion. The goal of coscheduling algorithms is to execute processes from the same job concurrently rather than maximize processor affinity.[136] There are several coscheduling implementations—matrix, continuous and

undivided. We present only the undivided algorithm, because it corrects some of the deficiencies of the matrix and continuous algorithms.

The **undivided coscheduling algorithm** places processes from the same job in adjacent entries in the global run queue (Fig. 15.13). The scheduler maintains a "window" equal to the number of processors in the system. All processes within a window execute in parallel for at most one quantum. After scheduling a group of processes, the window moves to the next group of processes, which also execute in parallel for one quantum. To maximize processor utilization, if a process in the window is suspended, the algorithm extends the sliding window one process to the right to allow another runnable process to run for the given quantum, even if it is not part of a job that is currently executing.

Because coscheduling algorithms use a round-robin strategy, they prevent indefinite postponement. Furthermore, because processes from the same job often run at the same time, coscheduling algorithms allow programs that are designed to run in parallel to take advantage of a multiprocessing environment. Unfortunately, the processor on which a process might be dispatched to a different processor each time, which can reduce processor affinity.[137]

Dynamic Partitioning

Dynamic partitioning minimizes the performance penalty associated with cache misses by maintaining high processor affinity.[138] The scheduler evenly distributes processors in the system among jobs. The number of processors allocated to a job is always less than or equal to the job's number of runnable processes.

For example, consider a system that contains 32 processors and executes three jobs—the first with eight runnable processes, the second with 16 and the third with 20. If the scheduler were to divide the processors evenly between jobs, one job would receive 10 processors and the other two jobs 11. In this case, the first job has only eight processes, so the scheduler assigns eight processors to that job and evenly distributes the remaining 24 processors (12 each) to the second and third jobs (Example 1 in Fig. 15.14). Therefore, a particular job always executes on a certain

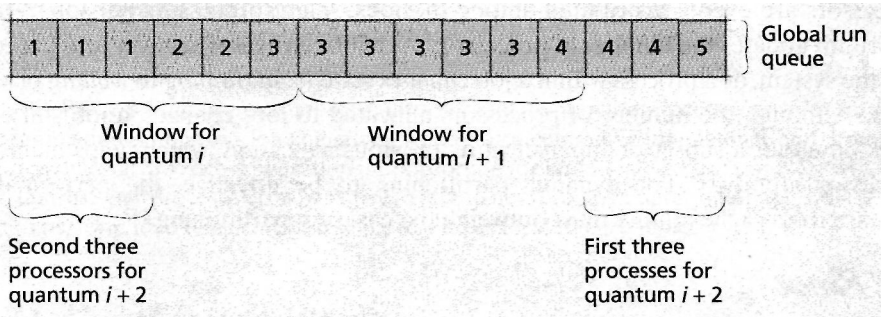

Figure 15.13 | *Coscheduling (undivided version).*

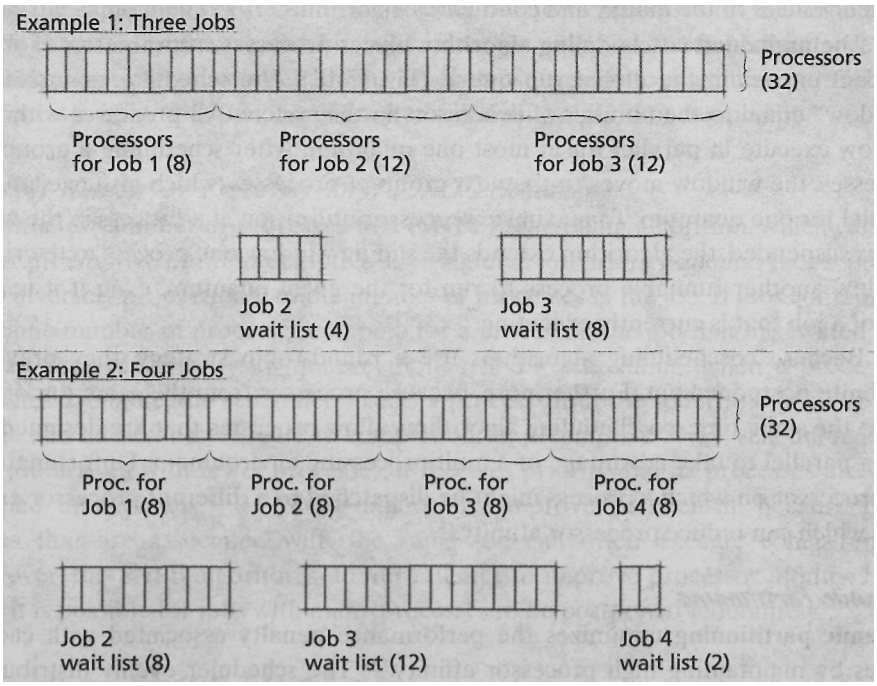

Figure 15.14 | *Dynamic partitioning.*

subset of processors as long as no new jobs enter the system. The algorithm can be extended so that a particular process always executes on the same processor. If every job contains only one process, dynamic partitioning reduces to a round-robin scheduling algorithm.[139]

As new jobs enter the system, the system dynamically updates the processor allocation. Suppose a fourth job containing 10 runnable processes (Example 2 in Fig. 15.14) enters the system. The first job retains its allocation of eight processors, but the second and third jobs each relinquish four processors to the fourth job. Thus, the processors are evenly distributed among the jobs—eight processors per job.[140] The algorithm updates the number of processors each job receives whenever jobs enter or exit the system, or a process within a job changes state from *running* to *waiting* or vice versa. Although the number of processors allocated to jobs changes, a job still executes on either a subset or superset of its previous allocation, which helps maintain processor affinity.[141] For dynamic partitioning to be effective, the performance increase from cache affinity must outweigh the cost of repartitioning.[142]

Self Review

1. How are RRJob and undivided coscheduling similar? How are they different?
2. Describe some of the trade-offs between implementing a global scheduling policy that maximizes processor affinity, such as dynamic partitioning, and per-processor run queues.

Ans: **1)** RRJob and undivided coscheduling are similar in that both schedule processes of a job to execute concurrently in round-robin fashion. Undivided coscheduling simply places processes of the same job next to each other in the global run queue where they wait for the next available processor. RRJob schedules only entire jobs. **2)** Global scheduling is more flexible because it reassigns processes to different processors depending on system load. However, per-processor run queues are simpler to implement and can be more efficient than maintaining global run queue information.

15.7 Process Migration

Process **migration** entails transferring a process between two processors.[143]-[144] This might occur if, for example, a processor fails or is overloaded.

The ability to execute a process on any processor has many advantages. The most obvious is that processes can move to processors that are underutilized to reduce process response times and increase performance and throughput.[145]-[146] (We describe this technique, called load balancing, in more detail in Section 15.8, Load Balancing.) Process migration also promotes fault tolerance.[147] For example, consider a program that must perform intensive, uninterrupted computation. If the machine running the program needs to be shut down or becomes unstable, the program's progress might be lost. Process migration allows the program to move to another machine to continue computation perhaps in a more stable environment.

In addition, process migration promotes resource sharing. In large-scale systems, some resources might not be replicated at every node. For example, in a NORMA system, processes might execute on machines with different hardware device support. A process might require access to a RAID array that is available through only one computer. In this case, the process should migrate to the computer with access to the RAID array for better performance.

Finally, process migration improves communication performance. Two processes that communicate frequently should execute on or near the same node to reduce communication latency. Because communication links are often dynamic, process migration can be used to make process placement dynamic.[148]

Self Review

1. What are some benefits provided by process migration?
2. On which types of systems (UMA, NUMA or NORMA) is process migration most appropriate?

Ans: **1)** Process migration promotes fault tolerance by moving processes away from malfunctioning nodes, supports load balancing, can reduce communication latency and promotes resource sharing. **2)** Process migration is appropriate for large-scale NUMA or NORMA systems that use per-processor (or per-node) run queues. Process migration enables these systems to perform load balancing and share resources local to each node.

15.7.1 Flow of Process Migration

Although process migration implementations vary across architectures, many implementations follow the same general steps (Fig. 15.15). First, a node issues a

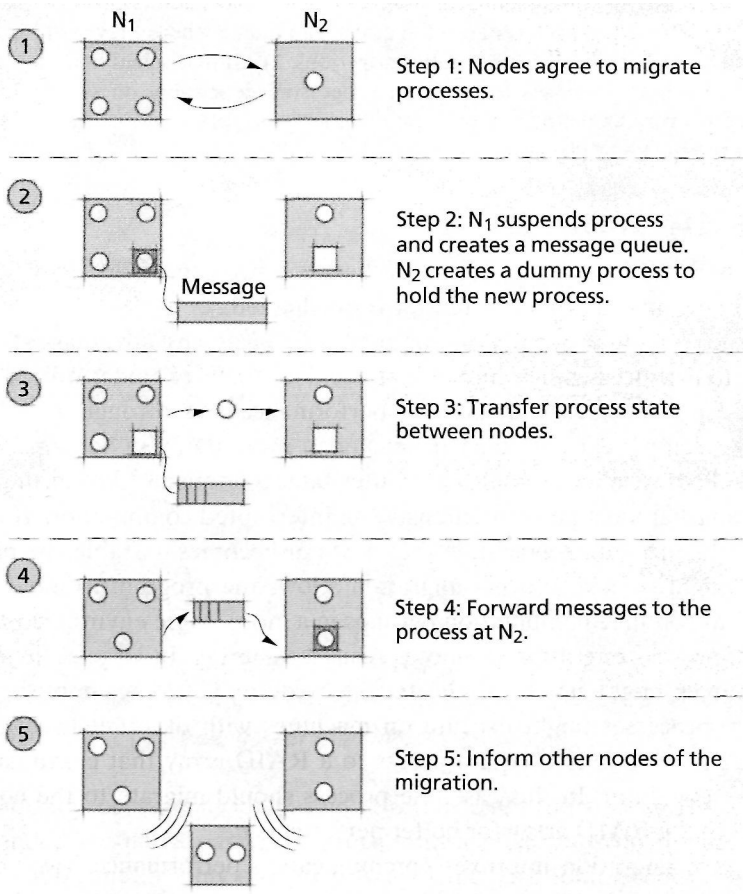

Figure 15.15 | *Process migration.*

migration request to a remote node. In most schemes, the sender initiates the migration because its node is overloaded or a specific process needs to access a resource located at a remote node. In some schemes, an underutilized node might request processes from other nodes. If the sender and receiver agree to migrate a process, the sender suspends the migrating process. The sender creates a message queue to hold all messages destined for the migrating process. Then, the sender extracts the process's state; this includes copying the process's memory contents (i.e., pages marked as valid in the process's virtual memory), register contents, state of open files and other process-specific information. The sender transmits the extracted state to a "dummy" process that the receiver creates. The two nodes notify all other processes of the migrated process's new location. Finally, the receiver dispatches the new instance of the process, the sender forwards the mes-

sages in the migrated process's message queue and the sender destroys the local instance of the process.[149]-[150]

Self Review

1. List some items that compose a process's state and that must be migrated with a process.
2. What overhead is incurred in process migration?

Ans: **1)** The sending process must transfer such items as the process's memory contents, register contents and state of open files. **2)** The overhead includes migrating the process and its state, rerouting messages to the receiving node, maintaining two instances of the process for a short time (the old instance and the "dummy process"), suspending the process for a short time and sending messages to other nodes to inform them about the migrated process's new location.

15.7.2 Process Migration Concepts

Memory transfer is the most time-consuming element of migration.[151] To minimize the process migration's performance cost, **residual dependency**—a process's dependency on its former node—must be minimized. For example, the process's former node might contain part of the process's working set or might be executing other processes with which the migrated one was communicating. If migration results in many residual dependencies, a receiving node must communicate with the sending node while executing the migrated process. This increases IPC, which generates more traffic on the interconnection network and degrades performance due to high latencies. Also, residual dependencies decrease fault tolerance, because the migrated process's execution depends on both nodes functioning correctly.[152]

Often, the strategies that result in the highest degree of residual dependency transfer pages from the sender only when the process at the receiving node references them. These **lazy** (or **on-demand) migration** strategies reduce the initial process migration time—the duration for which the migrating process is suspended. In lazy migration, the process's state (and other required) information is transferred to the receiver, but the original node retains the process's pages. As the process executes on the remote node, it must initiate a memory transfer for each access to a page that remains at the sending node. Although this technique yields fast initial process migration, memory access can severely decrease an application's performance. Lazy migration is most useful when the process does not require frequent access to the remote address space.[153]-[154]

For successful migration, processes should exhibit several characteristics. A migrated process should exhibit transparency—i.e., it should not be adversely affected (except, perhaps, for a slight delay in response time) by migration. In other words, the process should lose no interprocess messages or open file handles.[155] A system also must be scalable—if process's residual dependencies grow with each migration, then a system could quickly be overwhelmed by network traffic as processes request remote pages. Finally, advances in communication technology across multiple architectures have created the need for heterogeneous process migra-

tions—processes should be able to migrate between two different processor architectures in distributed systems. This implies that process state should be stored in a platform-independent format.[156] Section 17.3.6, Process Migration in Distributed Systems, discusses heterogeneous process migration architectures in more detail.

Self Review

1. Why is residual dependency undesirable? Why might some residual dependency be beneficial?
2. How might a migration strategy that results in significant residual dependency not be scalable?

Ans: **1)** Residual dependency is undesirable because it decreases fault tolerance and degrades performance after the transfer. One reason for allowing residual dependency is that it reduces initial transfer time. **2)** If a process that already has a residual dependency migrates again, it will now rely on three nodes. With each migration, the process's state grows (so it can find its memory pages) and its fault tolerance decreases.

15.7.3 Process Migration Strategies

Process migration strategies must balance the performance penalty of transferring large amounts of process data with the benefit of minimizing a process's residual dependency. In some systems, designers assume that most of a migrated process's address space will be accessed at its new node. These systems often implement **eager migration,** which transfers all of a process's pages during initial process migration. This enables the process to execute as efficiently at its new node as it did at its previous one. However, if the process does not access most of its address space, the initial latency and the bandwidth required by eager migration are largely overhead.[157, 158]

To mitigate eager migration's initial cost, **dirty eager migration** transfers only a process's dirty pages. This strategy assumes that there is common secondary storage (e.g., a disk to which both nodes have access). All clean pages are brought in from the common secondary storage as the process references them from the remote node. This reduces initial transfer time and eliminates residual dependency. However, each access to a nonresident page takes longer than it would have using eager migration.[159]

One disadvantage of dirty eager migration is that the migrated process must use secondary storage to retrieve the process's clean pages. **Copy-on-reference migration** is similar to dirty eager migration, except that the migrated process can request clean pages from either its previous node or the common secondary storage. This strategy has the same benefits as dirty eager migration but gives the memory manager more control over the location from which to request pages—access to remote memory may be faster than disk access. However, copy-on-reference migration can add memory overhead at the sender.[160]-[161]

The **lazy copying** implementation of copy-on-reference transfers only the minimum state information at initial migration time; often, no pages are transferred. This creates a large residual dependency, forces the previous node to keep pages in memory for the migrated process and increases memory-access latency over strategies that migrate dirty pages. However, the lazy copying strategy eliminates most

initial migration latency.[162, 163] Such latency might be unacceptable for real-time processes and is inappropriate for most processes.[164]

All the strategies discussed so far either create residual dependency or incur a large initial migration latency. A method that eliminates most of the initial migration latency and residual dependency is the **flushing** strategy. In this strategy, the sender writes all pages of memory to shared secondary storage when migration begins; the migrating process must be suspended while the data is being written to secondary storage. The process then accesses the pages from secondary storage as needed. Therefore, no actual page migration occurs to slow initial migration, and the process has no residual dependency on the previous node. However, the flushing strategy reduces process performance at its new node, because the process has no pages in main memory, leading to page faults.[165-166]

Another strategy for eliminating most of the initial migration latency and residual dependency is the **precopy** method, in which the sender node begins transferring dirty pages before the original process is suspended. Any transferred page that the process modifies before it migrates is marked for retransmission. To ensure that a process eventually migrates, the system defines a lower threshold for the number of dirty pages that should remain before the process migrates. When that threshold is reached, the process is suspended and migrated to another processor. With this technique, the process does not need to be suspended for long (compared to other techniques such as eager and copy-on-reference), and memory access at the new node is fast because most data has already been copied. Also, residual dependency is minimal. The process's working set is duplicated for a short time (it exists at both nodes involved in migration), but this a minor drawback.[167168]

Self Review

1. Which migration strategies should be used with real-time processes?
2. Although initial migration time is minimal and there is little residual dependency for the precopy strategy, can you think of any "hidden" performance costs this strategy incurs?

Ans: **1)** Real-time processes cannot be suspended for long because this would slow response time. Therefore, strategies such as lazy copying, flushing, copy-on-reference and precopy are best for soft real-time processes. However, hard real-time processes should not be migrated because migration always introduces some indeterministic delay. **2)** The process must continue executing on the sender node while its state is being copied to the receiving node. Because a migration decision was made, it is reasonable to assume that executing on the sending node is no longer desirable and therefore, precopy includes this additional cost.

15.8 Load Balancing

One measure of efficiency in a multiprocessor system is overall processor utilization. In most cases, if processor utilization is high, the system is performing more efficiently. Most multiprocessor systems (especially NUMA and NORMA systems) attempt to maximize processor affinity. This increases efficiency, because the processes do not need to access remote resources as often, but it might reduce processor

utilization if all the processes assigned to a particular processor complete. This processor will idle while processes are dispatched to other processors to exploit affinity. **Load balancing** is a technique in which the system attempts to distribute processing loads evenly among processors. This increases processor utilization and shortens run queues for overloaded processors, reducing average process response times.[169]

A load balancing algorithm might assign a fixed number of processors to a job when the job is first scheduled. This is called **static load balancing.** This method yields low runtime overhead, because processors spend little time determining the processors on which a job should execute. However, static load balancing does not account for varying process populations within a job. For example, a job might include many processes initially, but maintain only a few processes throughout the rest of its execution. This can lead to unbalanced run queues, which may lead to processor idling.[170]

Dynamic load balancing attempts to address this issue by adjusting the number of processors assigned to a job throughout its life. Studies have shown that dynamic load balancing performs better than static load balancing when context-switch time is low and system load is high.[171]

Self Review

1. What are some of the benefits of implementing load balancing?
2. What scheduling algorithms benefit from load balancing?

Ans: **1)** Some of the benefits of load balancing include higher processor utilization, which leads to higher throughput, and reduced process response time. **2)** Load balancing is appropriate for scheduling algorithms that maximize processor affinity, such as dynamic partitioning. Also, it can benefit systems that maintain per-processor or per-node run queues.

15.8.1 Static Load Balancing

Static load balancing is useful in environments in which jobs repeat certain tests or instructions and therefore exhibit predictable patterns (e.g., scientific computing).[172] These patterns can be represented as graphs that can be used to model scheduling. Consider the processes in a system as vertices in a graph, and communications between processes as edges. For example, if there is an application in which one process continually analyzes similar data, then feeds that data to another process, this can be modeled as two nodes connected by one edge. Because this relationship will be consistent throughout the application's life, there is no need to adjust the graph.

Due to shared cache and physical memory, communication between processes at the same processor is much faster than communication between processes at different processors. Therefore, static load balancing algorithms attempt to divide the graph into subgraphs of similar size (i.e., each processor has a similar number of processes) while minimizing edges between subgraphs to reduce communication between processors.[173] However, this technique can incur significant overhead for a large number of jobs.[174] Consider the graph in Fig. 15.16. Two dashed lines represent

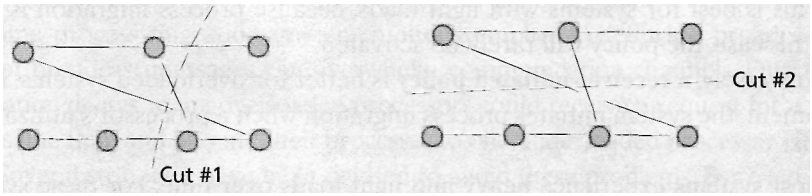

Figure 15.16 | *Static load balancing using graphs.*

possible cuts to divide the processes somewhat evenly. Cut #1 yields four interprocessor communication channels, whereas cut #2 yields only two, thus representing a better grouping of processes.

Static load balancing can be inadequate when communication patterns change dynamically and when processes complete with no new processes to take their places. The first case might perform less efficiently due to high communication latency. In the second case, processor utilization could decrease even when there are processes waiting to obtain a processor. In these cases, dynamic load balancing can improve performance.[175]

Self Review

1. When is static load balancing useful? When is it not useful?
2. In Fig. 15.16, there is a substantial difference between cut #1 and cut #2. Considering that it is difficult to find the most effective cut when there are large numbers of jobs, what does this say about the limitations of static load balancing?

Ans: **1)** Static load balancing is useful in environments where processes might exhibit predictable communication patterns. It is not useful in environments where communication patterns change dynamically and processes are created or terminated unpredictably. **2)** The difference between Cut #1 (four IPC channels) and Cut #2 (two IPC channels) illustrates the performance implications of making bad decisions. Most large systems will have to estimate to find a solution; a wrong estimate could severely degrade performance.

15.8.2 Dynamic Load Balancing

Dynamic load balancing algorithms migrate processes after they have been created in response to system load. The operating system maintains statistical processor load information such as the number of active and blocked processes at a processor, average processor utilization, turnaround time and latency.[176-177] If many of a processor's processes are blocked or have a large turnaround times, the processor most likely is overloaded. If a processor does not have a high processor utilization rate, it probably is underloaded.

Several policies may be used to determine when to migrate processes in dynamic load balancing. The **sender-initiated policy** activates when the system determines that a processor contains a heavy load. Only then will the system search

for underutilized processors and migrate some of the overloaded processor's jobs to them. This is best for systems with light loads, because process migration is costly and, in this case, the policy will rarely be activated.[178, 179]

Conversely, a **receiver-initiated policy** is better for overloaded systems. In this environment, the system initiates process migration when a processor's utilization is low.[180, 181]

Most systems experience heavy and light loads over time. For these systems, the **symmetric policy,** which combines the previous two methods, provides maximum versatility to adapt to environmental conditions.[182, 183] Finally, the **random policy,** in which the system arbitrarily chooses a processor to receive a migrated process, has shown decent results due to its simple implementation and (on average) even distribution of processes. The motivation behind the random policy is that the migrating process's destination will likely have a smaller load than its origin, considering that the original processor is severely overloaded.[184, 185]

The subsections that follow describe common algorithms that determine how processes are migrated. For the purposes of this discussion, consider that the multiprocessor system can be represented by a graph in which each processor and its memory are a vertex and each link is an edge.

Bidding Algorithm

The **bidding algorithm** is a simple sender-initiated migration policy. Processors with smaller loads "bid" for processes on overloaded processors, much as in an auction. The value of a bid is based on the current load of the bidding processor and the distance between the underloaded and overloaded processors in the graph. To reduce the process migration cost, more direct communication paths to the overloaded processor receive higher bid values. The overloaded processor accepts bids from processors that are within a certain distance on the graph. If the overloaded processor receives too many bids, it decreases the distance; if it receives too few, it increases the distance and checks again. The process is sent to the processor with the highest bid.[186]

Drafting Algorithm

The **drafting algorithm** is a receiver-initiated policy that classifies the load at each processor as low, normal or high. Each processor maintains a table describing the other processors' loads using these classifications. Often in large-scale or distributed systems, processors maintain only information about their neighbors. Even time a processor's load changes classification, the processor broadcasts its updated information to the processors in its load table. When a processor receives one of these messages, it appends its own information and forwards the message to its neighbors. In this way, information about load levels eventually reaches all nodes in the network. Underutilized processors use this information to request processes from overloaded processors.[187]